



UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko

Nevronske mreže

Praktikum strojnega učenja

Simon Čopar

Kaj je sploh nevronska mreža?

↳ „No free lunch“:

↳ **ni** magično boljša od ostalih metod za vse probleme

↳ **drag trening**: impresivni modeli so rezultat zelo velikih setov podatkov

↳ Stara ideja (McCulloch, Pitts, 1943)

↳ Teorija: Walter Pitts, Warren McCulloch, Donald Hebb

↳ Kmalu implementacije, 1965 že večplastno, 1975 praktičen algoritem

↳ Dolgo čakanje na „renesanso“ v 2010+ (Andrew Ng,...).

↳ Spet fitamo

↳ Tokrat iščemo neznano preslikavo (funkcijo) med visokodimenzijskima prostoroma

↳ Kako parametriziramo zvezno, nelinearno „univerzalno funkcijo“?

↳ Začnemo z linearno, „začinimo“ z nelinearnostjo

↳ Iščemo matrike

↳ Rekurzija (deep networks)

↳ Moč nevronske mreže je v zveznosti (zvezna odvedljivost, možnost posploševanja – interpolacija v N-dimenzijskem prostoru)

Nekaj zgodovine

- ↳ 1943: Warren McCulloch & Walter Pitts, teoretični modeli delovanja nevronov
- ↳ 1949: Donald Hebb in povezava z nevropsihologijo: Hebbian learning
- ↳ 1958: Frank Rosenblatt, poskusi hardwarskih implementacij „perceptrona“ (multilayer perceptron)
- ↳ 1982: John Hopfield
 - ↳ Boltzmann machine, Hopfield network
 - ↳ Ideja iz fizike spinskih mrež
 - ↳ Ni feed-forward ampak samokonsistentna mreža!
- ↳ 1970-1980: back-propagation, matematična osnova za izvedljiv trening
- ↳ 2000': večje baze, ponovno zanimanje za NN, nišna uporaba
 - ↳ LeCun: 1998 – CNN za prepoznavo števil; začetek renesanse nevronske mreže
- ↳ 2012-2020: dramatičen razvoj konvolucijskih nevronske mreže in NN na splošno
 - ↳ prehod iz akademskega sveta v praktično uporabo
 - ↳ Končno računalniki dohajajo (boljši CPU, računski GPU)
 - ↳ **Več podatkov, končno lahko treniramo ker imamo podatke!**
- ↳ 2017-: Transformerji, „**attention is all you need**“
 - ↳ Predhodnik sodobnih jezikovnih modelov (ChatGPT, osnova text-to-image modelov)
- ↳ 2024: Nobelova nagrada za fiziko in kemijo sta iz NN
 - ↳ Začetki nevronske mreže so v resnici osnovani na fizikalnih idejah

2024 Nobel Prize

2024 Nobel Prize in physics
awarded to John J. Hopfield,
Geoffrey E. Hinton for
discoveries that 'enable
machine learning with artificial
neural networks'



John J.
Hopfield



Geoffrey E.
Hinton



Nekaj zgodovine



UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko

THE NOBEL PRIZE IN PHYSICS 2024

Illustrations: Niklas Elmehed



John J. Hopfield

Geoffrey E. Hinton

"for foundational discoveries and inventions
that enable machine learning
with artificial neural networks"

THE ROYAL SWEDISH ACADEMY OF SCIENCES

THE NOBEL PRIZE IN CHEMISTRY 2024

Illustrations: Niklas Elmehed



**David
Baker**

**Demis
Hassabis**

**John M.
Jumper**

"for computational
protein design"

"for protein structure prediction"

THE ROYAL SWEDISH ACADEMY OF SCIENCES

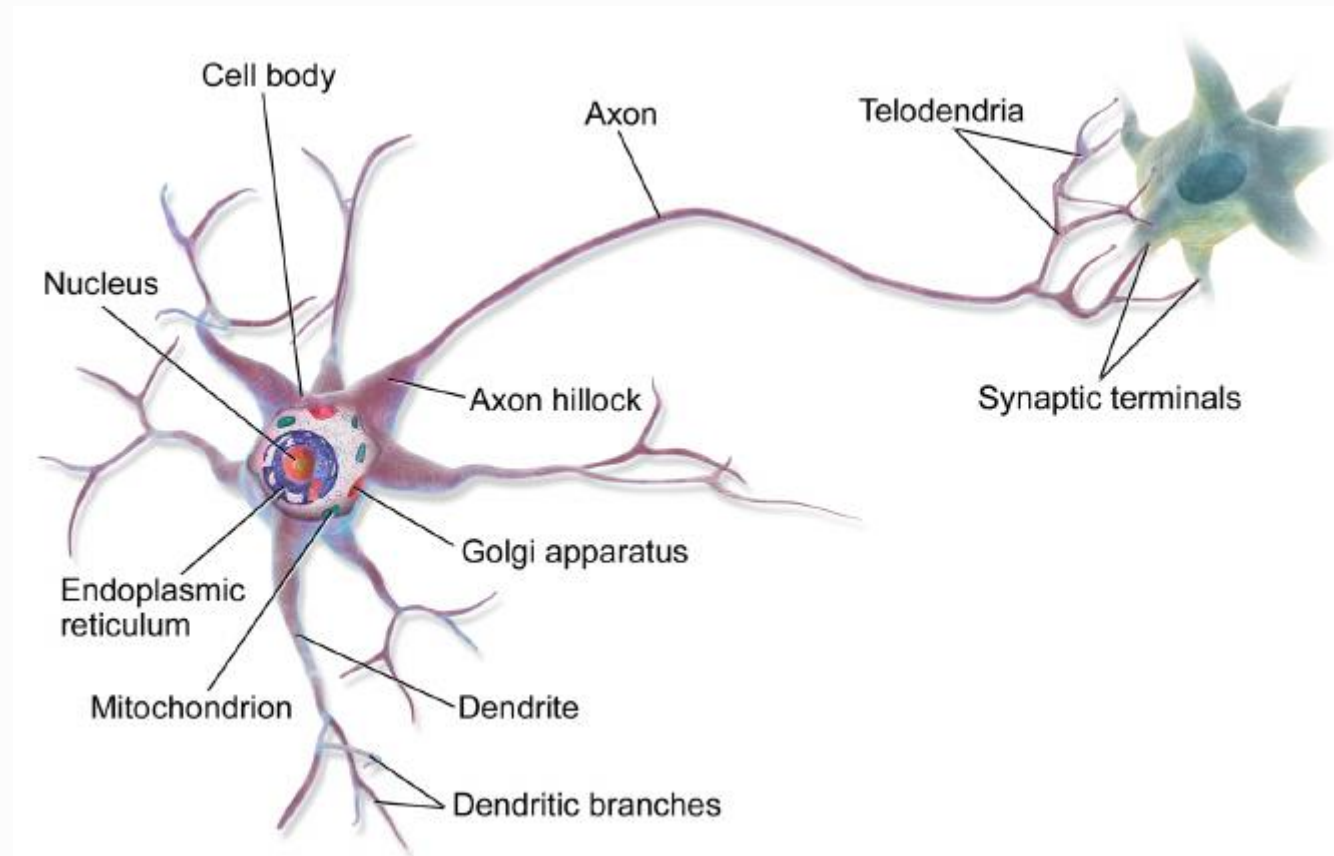
Motivacija: kako deluje živčevje?



UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko



Ponovitev: logistična regresija

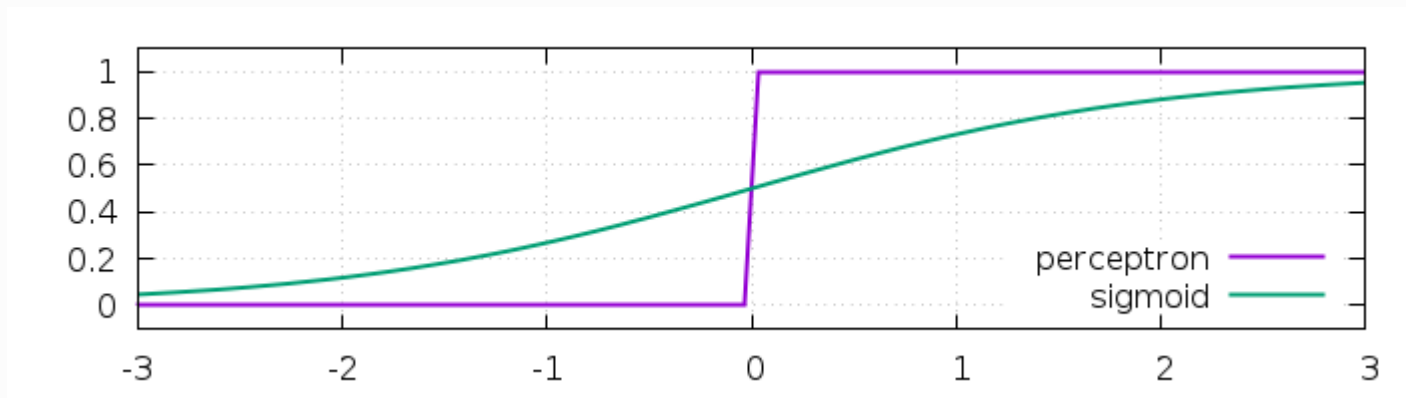
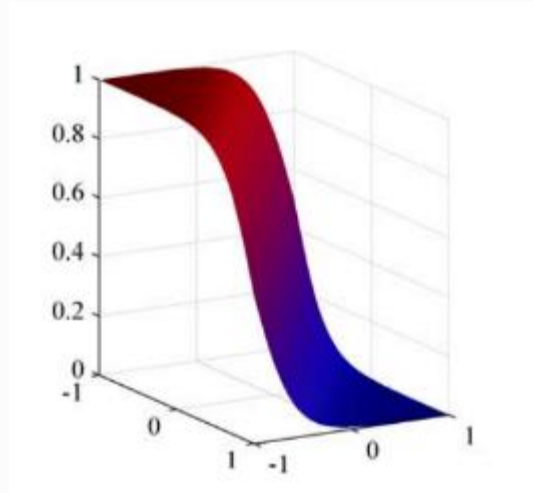
↳ Iskanje meje med kategorijami v m -dimenzionalnem prostoru

↳ Logistična regresija: $y = \sigma(\theta x)$, vključuje bias

↳ ML (maximum likelihood), gradientni spust:

$$\theta \mapsto \theta + \eta(t_i - \sigma(\theta x_i))x_i$$

learning rate podatki izhod modela (verjetnost) vektor uteži



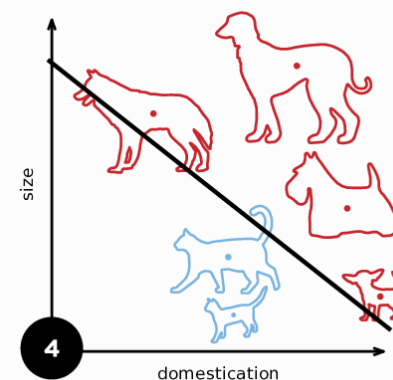
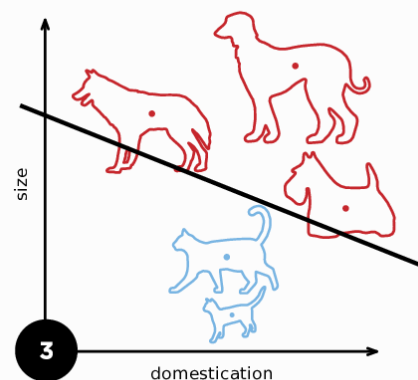
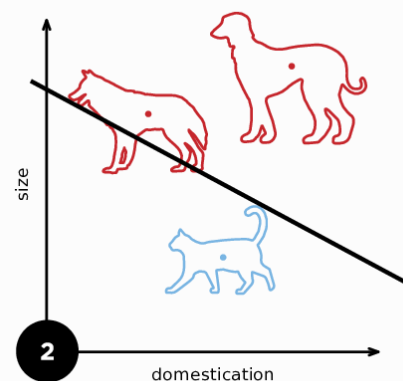
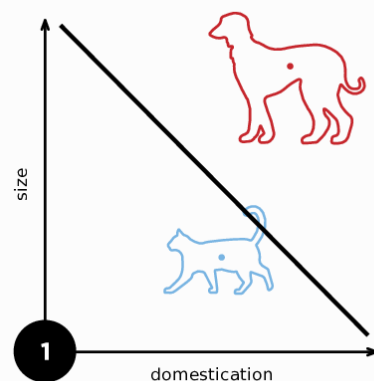
Ponovitev: logistična regresija



FMF

UNIVERZA
V LJUBLJANI

Fakulteta za matematiko
in fiziko



Perceptron



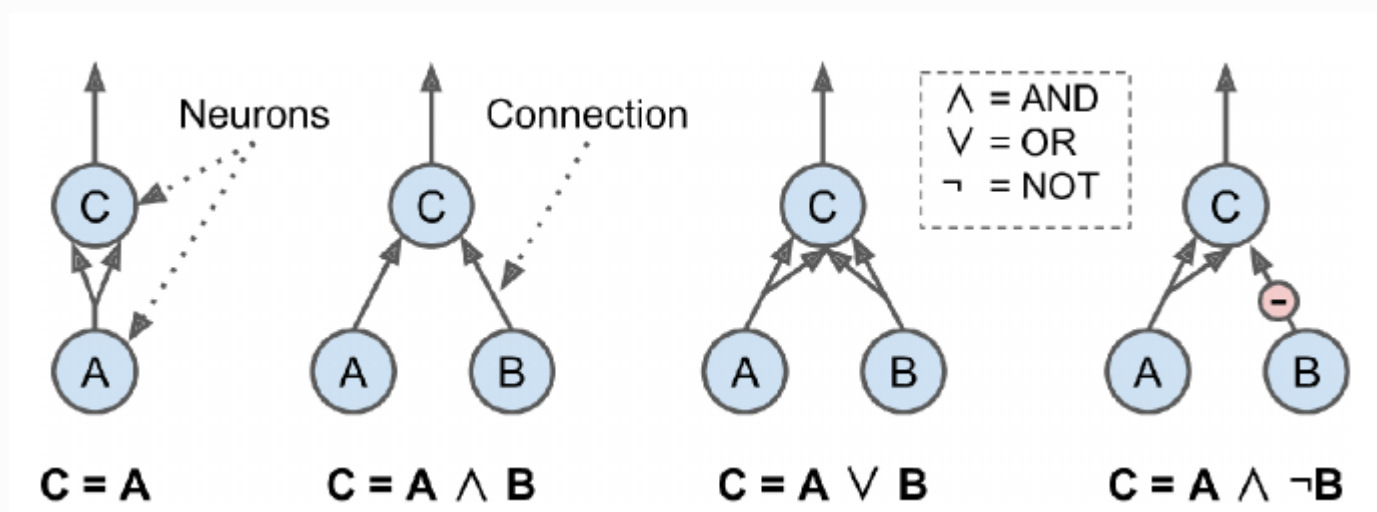
UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko

McCulloch & Pitts, 1943

- ↳ Najenostavnejši model: binarna aktivacija → 0 do 1
- ↳ Iz perceptronov lahko sestavimo logične (Boolove) operacije
- ↳ Torej: upamo, da bo naša funkcija sposobna sklepanja



- ↳ Gradientni spust enako kot logistična regresija:

$$\theta \mapsto \theta + \eta(t_i - f(\theta x_i))x_i$$

Plast perceptronov

↳ Več vprašanj: Ima ušesa? Je kosmat? Je maček? Je pes? Je avto? Ima veliko vogalov? Je črka?

↳ Vektor logističnih regresij oz. perceptronov

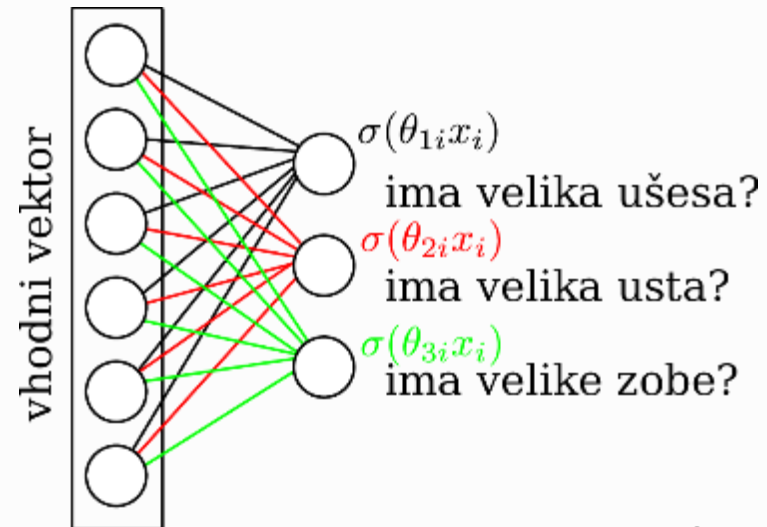
↳ **Ideja:** iz več vprašanj lahko izvemo več

$$y_j = f(\theta_j x) \Rightarrow \mathbf{y} = f(W\mathbf{x})$$

aktivacijska funkcija
(nelinearna)

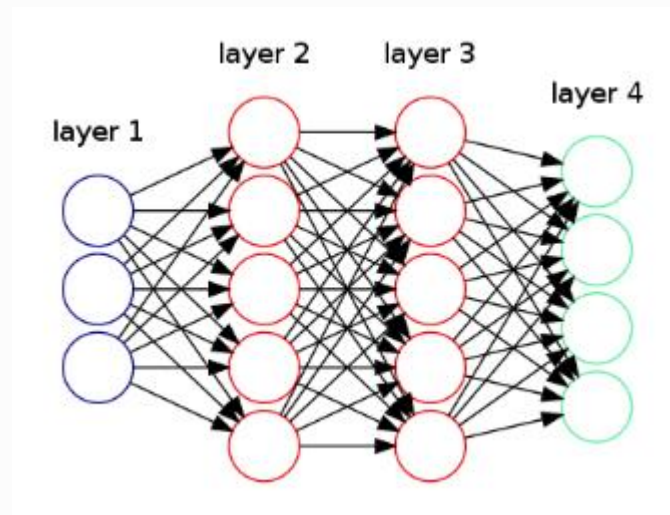
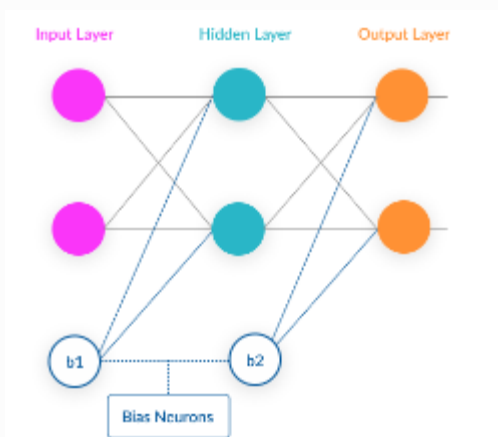
matrika uteži
(weights)

↳ **Problem:** trening za vse te značilke?



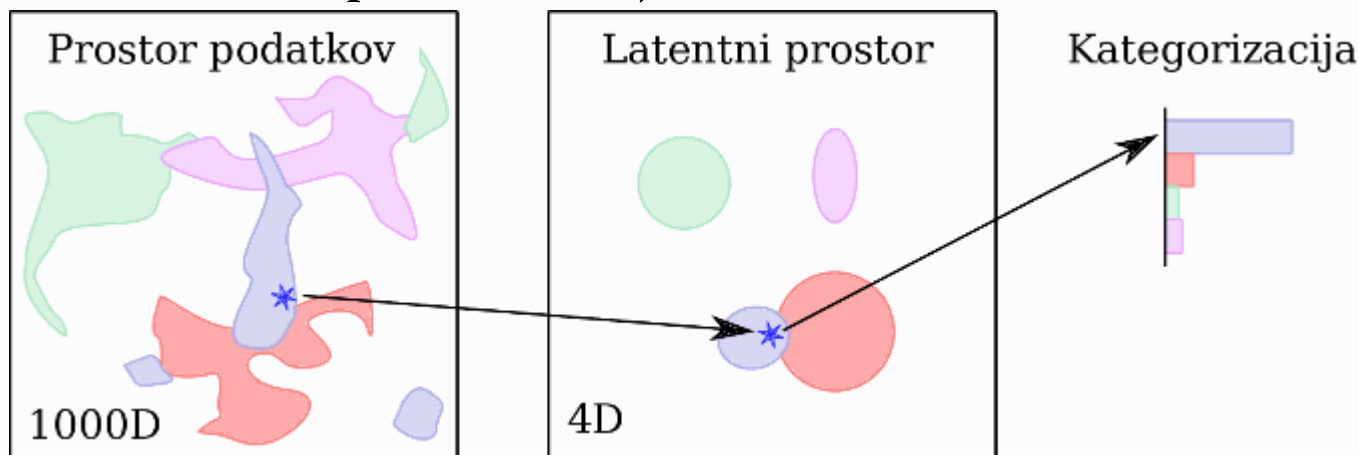
Skrite plasti

- ↳ Vhodni prostor je kompleksen, ni separabilen niti s Kernel trikom
- ↳ **Hierarhija**: odgovor gradimo po korakih
- ↳ Kompozicija več plasti perceptronov
- ↳ Rezultat prve plasti so značilke (features) za naslednjo plast
- ↳ **Upamo**: vmesna plast se sama nauči, kaj so pametne lastnosti
- ↳ Ne pozabimo: v resnici imamo vedno prisoten konstantni člen (bias)



Skrite plasti

- ↳ Običajni ML: prekletstvo dimenzij
- ↳ NN: dodatne plasti „pregnetejo“ vhod v boljše značilke, generirajo še več podatkov, a z boljšo strukturo
- ↳ Končni učinek običajno spet redukcija dimenzij
 - ↳ Izjeme – generativni modeli (iz malo v veliko podatkov!)
- ↳ Posledica: uteži je lahko več kot vhodnih podatkov, poddoločen sistem
 - ↳ Uteži ne razumemo – ne vemo, katere so pomembne in kaj računajo
 - ↳ Drag trening: veliko uteži zahteva več vhodnih podatkov
 - ↳ Iztisniti čim več iz podatkov, ki jih imamo



Skrite plasti

↳ Koliko nevronov na plast potrebujemo?

↳ Tipične oblike:

↳ Postopno ožanje do končne velikosti (enkoder)

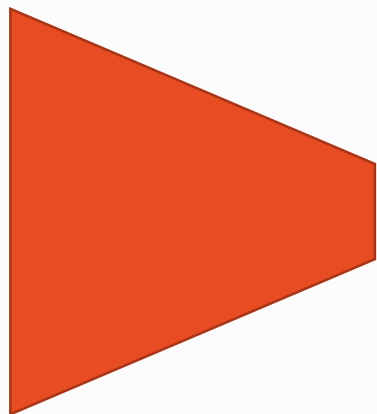
↳ Zožitev + ozko grlo + razširitev

↳ Razširitev (generiranje podatkov, dekodek)

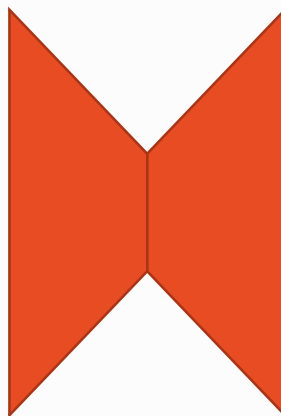
↳ Primerjaj: „quench/anneal“ kako hitro prisilimo redukcijo dimenzij

↳ Koliko plasti potrebujemo?

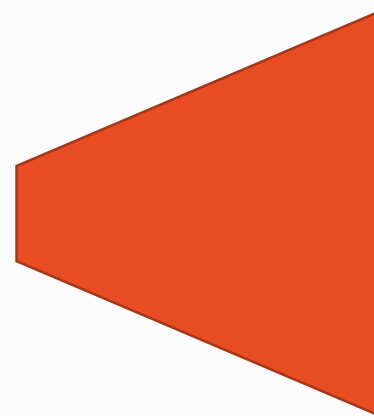
↳ Koliko nivojev kompleksnosti ima problem?



enkoder / klasifikacija /
regresija



avtoenkoder



dekoder / generator

Izrek o univerzalni aproksimaciji

↳ S kompozicijo dovolj velike količine **nelinearnih** preslikav lahko vsako funkcijo aproksimiramo poljubno natančno

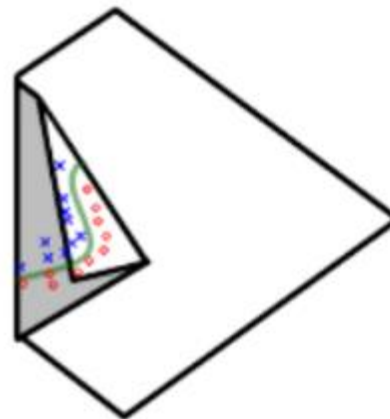
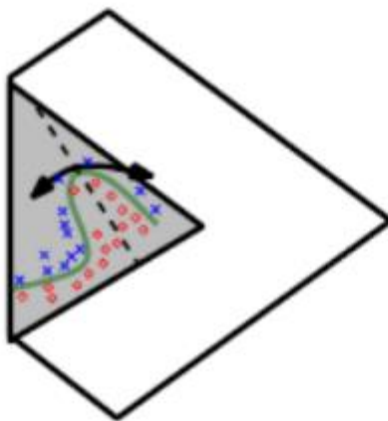
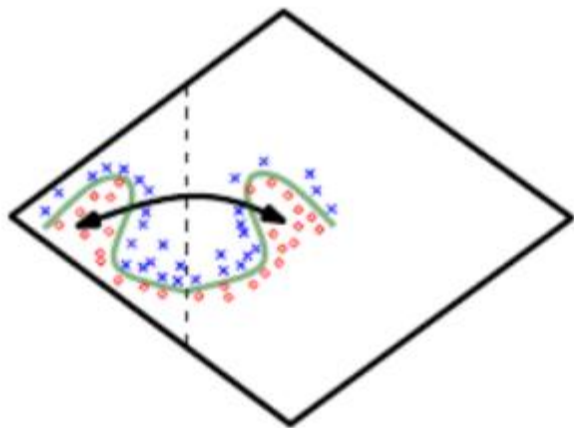
↳ Če je **linearna**, kolapsira na eno preslikavo:

$$\mathbf{y} = \mathbf{W}^{(3)}\mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x} = \mathbf{W}'\mathbf{x}$$

Slabo – nič ne pridobimo

↳ „Deep Neural Network“ – veliko plasti

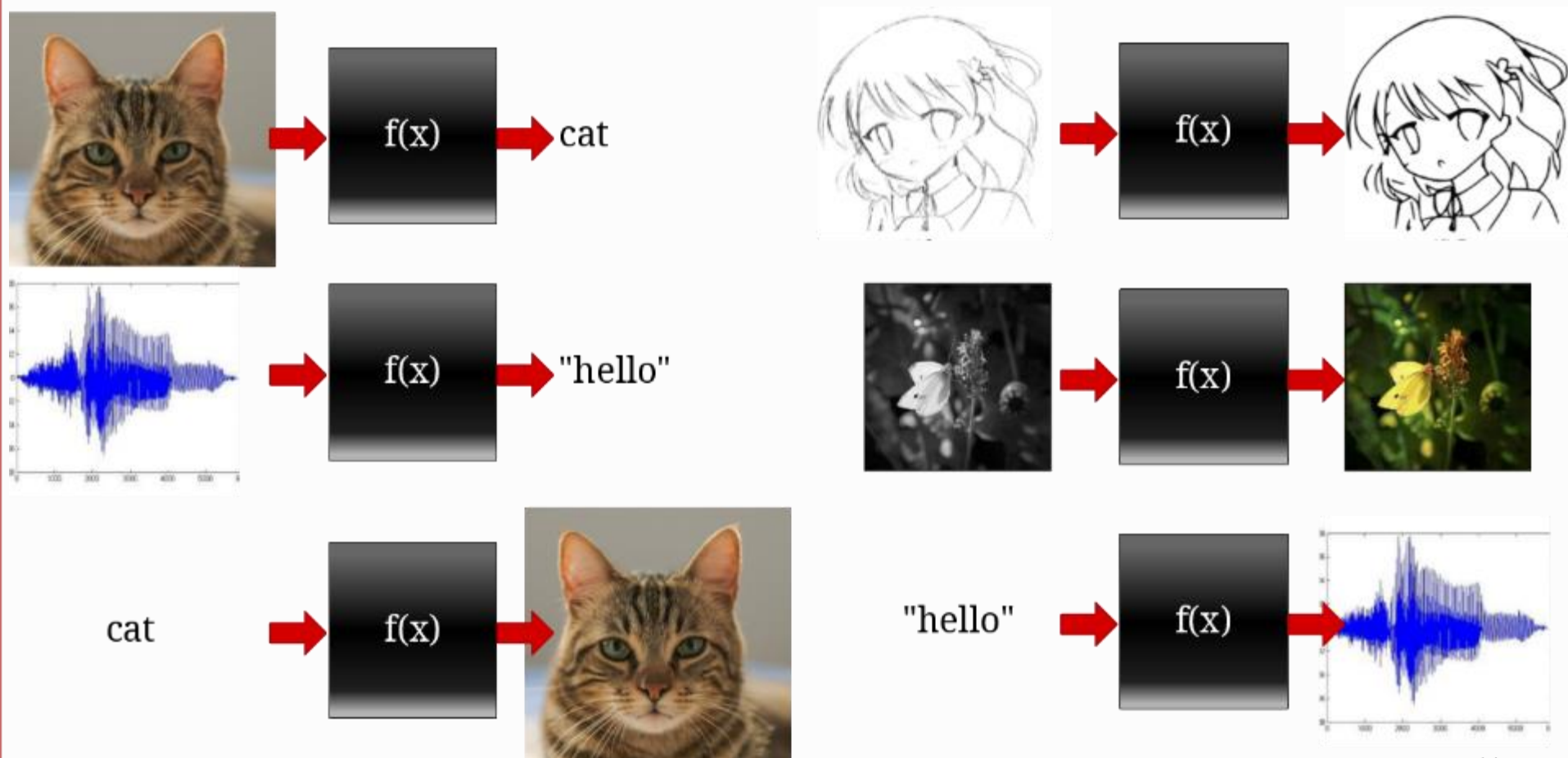
↳ Težje minimizirat, potrebuje več podatkov, numerični problemi



Kaj mislimo s pojmom univerzalna funkcija?

↳ Slikamo katerikoli objekt v katerikoli rezultat

↳ Matematično: vse je lahko „vektor“



Nevronske mreže



UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko

↳ Plasti nevronov: kompozicija matričnih množenj in nelinearnih funkcij

$$\mathbf{y} = f \left(W^{(3)} f \left(W^{(2)} f \left(W^{(1)} \mathbf{x} \right) \right) \right)$$

Cilj minimizacije

Prva plast

↳ Iščemo uteži W , da bo izhod $\mathbf{y}$ čim bližje **znanemu cilju $\mathbf{t}$** . target

↳ Minimiziramo „Loss function“
 $J_{W^{(j)}}(\mathbf{y}, \mathbf{t})$ z minimumom pri $\mathbf{y} = \mathbf{t}$

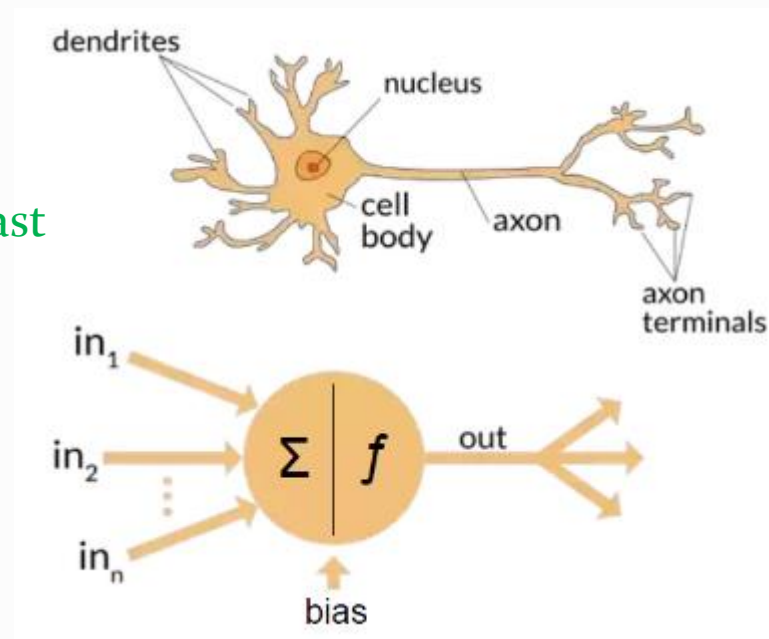
↳ Analogija z nevroni:

- ↳ Nevron aktivira vsota vhodnih sinaps
- ↳ Aktivacijska funkcija normalizira izhod
- ↳ Izkaže se, da ta normalizacija ni vedno dobra

Matrične operacije $\Rightarrow$ hitra evaluacija, GPU

Vsa nelinearnost je v aktivacijski funkciji f , ki se računa na vektorju – ni kernela

Nobenih inverzov in drugih $\mathcal{O}(n^3)$ operacij



Bias (o-ta komponenta) je tudi prisoten in predmet minimizacije

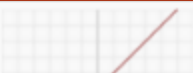
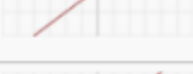
Aktivacijske funkcije (activation function = link function)



FMF

Fakulteta za matematiko
in fiziko

UNIVERZA
V LJUBLJANI

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
Tanh		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

Aktivacijske funkcije

Ime	Funkcija	Značilnosti	Prednosti	Slabosti
ReLU	$\max(0, x)$	Pozitivna, neomejena	Hiter Ni saturacije	Mrtvi nevroni (ničelni gradienti) Nezvezen odvod
Softplus	$\log(1 + e^x)$	Gladek ReLU	Zvezen odvod	Neuravnovešeni predznaki
Sigmoid	$\frac{1}{1 + e^{-x}}$	Od 0 do 1	Interpretacija kot verjetnost	Saturacija (izginuli gradienti) Neuravnovešeni predznaki
Tanh	$\tanh x$	Od -1 do 1	Uravnovešanje predznakov	Kot sigmoid
ELU, Parametric ReLU	$\max(ax, x)$	ReLU s puščanjem	Ni mrtvih nevronov	

Arhitektura nevronske mreže



UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko

↳ Zadnja plast regulira kaj je naš izhod!

↳ Razvrščanje v razrede (multiclass)

↳ samo ena enka, izhodi so „verjetnostna“
porazdelitev po razredih

↳ Določanje oznak (multilabel)

↳ Vsak izhod je „verjetnost“ za pravilnost oznake

↳ **Regresija**/feature extraction/enkoder

↳ Izhod je poljubna funkcija vhoda – število, vektor, slika

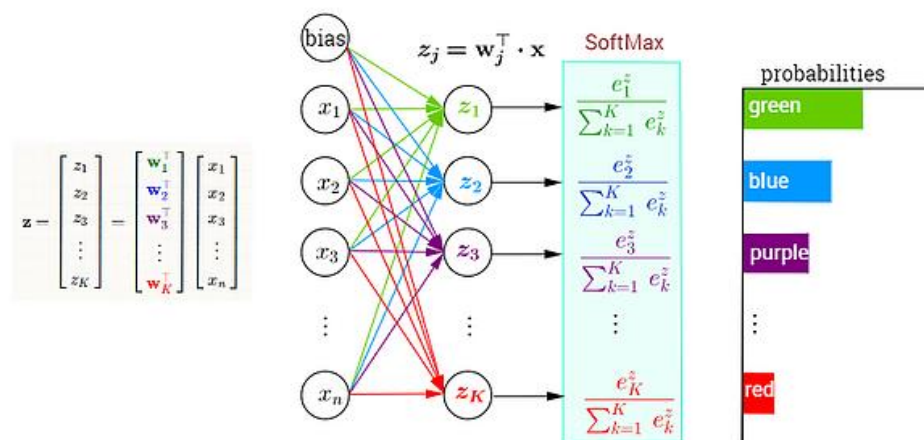
Checkboxes

- ☒ Option 1
- ☐ Option 2
- ☐ Option 3
- ☒ Option 4

Radio Buttons

- ☐ Option 1
- ☒ Option 2
- ☐ Option 3
- ☐ Option 4

Multi-Class Classification with NN and SoftMax Function



Arhitektura nevronske mreže

↳ Normalizacija vhodnih podatkov

↳ Da ne saturiramo vhoda in izkoristimo aktivacijsko funkcijo

↳ Preslikava izhodnih podatkov v željeno obliko

↳ Minimizacija funkcije izgube oz. cenovne funkcije („Loss function“)

Uporaba	Cenovna funkcija	Aktivacijska funkcija zadnje plasti
Regresija (realen output)	Vsota kvadratov napake	Glede na potrebe
Klasifikacija (verjetnost po razredih)	Navzkrižna entropija	Softmax (>0 , vsota = 1)
Označevanje (da/ne)	Navzkrižna entropija	Sigmoid (>0)

$$J(\mathbf{y}, \mathbf{t}) = \sum_i (y_i - t_i)^2$$

MSE (Mean squared error) – najmanjši kvadrati

$$J(\mathbf{y}, \mathbf{t}) = - \sum_i t_i \log y_i$$

Cross entropy – navzkrižna entropija

$$y_i = \frac{e^{\tilde{y}_i}}{\sum_i e^{\tilde{y}_i}}$$

lin. izhod zadnje matrike

Fiziki: statistična vsota,
 $\exp(-\beta E)$

Inicializacija nevronske mreže

↳ Vhodni podatki: **normalizacija**: $(-1,1)$, včasih $(0,1)$

↳ bias pomaga malo pretvarjat, ampak območje naj bo skladno z izhodom aktivacijske funkcije

↳ `sklearn.preprocessing.StandardScaler` (`sklearn/keras`)

↳ Začetni približek uteži: naključno Gaussovo ali enakomerno

↳ matrika $m \times n$, če ima vhodni vektor $\sigma^2 = 1$ in matrika σ_w^2 , hočemo na izhodu (naslednji vektor) $n \sigma_w^2 = 1$

↳ LeCun initialization: $\sigma_w^2 = \frac{1}{n}$

↳ He initialization: $\sigma_w^2 = \frac{2}{n}$

↳ Glorot initialization: $\sigma_w^2 = \frac{1}{(m+n)/2}$ ali enakomerno med $\pm \sqrt{\frac{3}{(m+n)/2}}$

Initialization	Activation functions	σ^2 (Normal)
Glorot	None, tanh, logistic, softmax	$1 / fan_{avg}$
He	ReLU and variants	$2 / fan_{in}$
LeCun	SELU	$1 / fan_{in}$

iz Gerona

Keras:

```
kernel_initializer=keras.initializers.GlorotNormal()  
.GlorotUniform()  
.HeUniform()
```

Mera uspešnosti

- ↳ Lahko gledamo kar funkcijo izgube
 - ↳ V teh primerih manj oprijemljivo, tudi odvisno od
 - ↳ Uporabno, kadar ne gre za klasifikacijo/kategorije
 - ↳ Razlika kvadratov

- ↳ Običajno opazujemo izkoristek

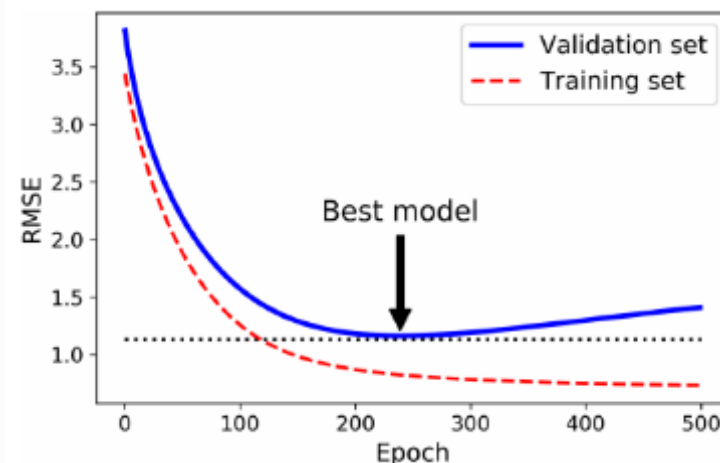
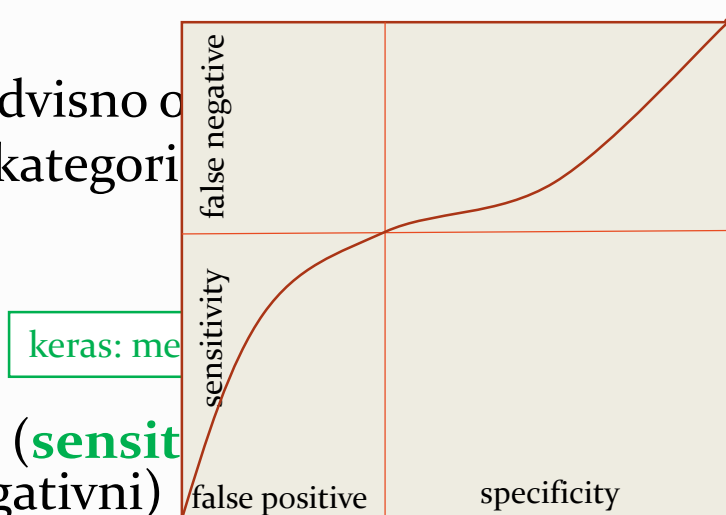
- ↳ Delež pravilno določenih (**accuracy**)
- ↳ Glede na potrebe lahko tudi občutljivost (**sensitivity**) ali specifičnost (**specificity**: pravilni negativni)

- ↳ **AUC** (Area Under the Curve)

- ↳ Za klasifikatorje

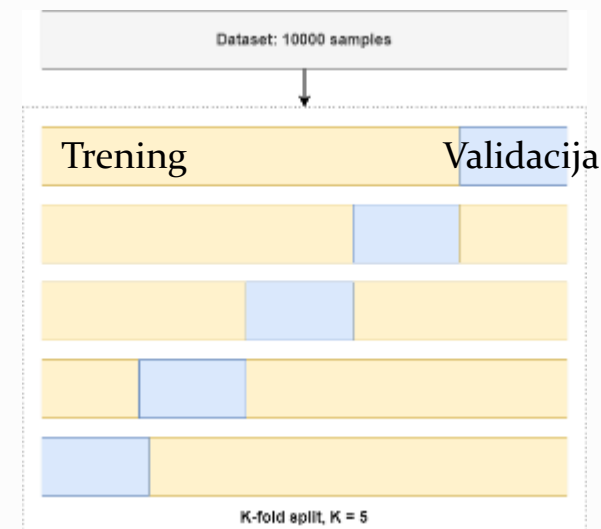
- ↳ Mero uspešnosti vedno izračunamo na **validacijskih** podatkih

- ↳ S tem zaznamo overfitting in končamo
- ↳ Mere uspešnosti pomagajo izbrati najboljši model



Učenje – minimizacija funkcije izgube

- ↳ Ogromno parametrov: potrebujemo veliko podatkov
- ↳ Minimizacija: običajno gradientni spust ali hitrejša metode
- ↳ **Epohe** – večkrat skozi cel set podatkov
 - ↳ Nevarnost overfitanja, ker ponavljamo podatke
 - ↳ vedno **training set** + **validation set** + **test set**
 - ↳ **trening**: za popravljanje uteži med treningom
 - ↳ **validacija**: za izračun izgube in metrik med treningom
 - ↳ **test**: čisto drug set za evaluacijo po treningu
 - ↳ Early stopping (podobno kot pri ostalih ML metodah)
- ↳ Navzkrižno preverjanje (cross-validation)
 - ↳ Najenostavnejši: K-fold treniramo K-krat **od začetka**
 - ↳ K-krat dražje, a bolje izkoristi podatke
 - ↳ **Povpreči** mero uspešnosti
 - ↳ **Izberi** najboljši model (ali treniraj še 1× čez vse)



```
sklearn.model_selection import KFold  
kfold=KFold(n_splits=6, shuffle=True)
```


Primer

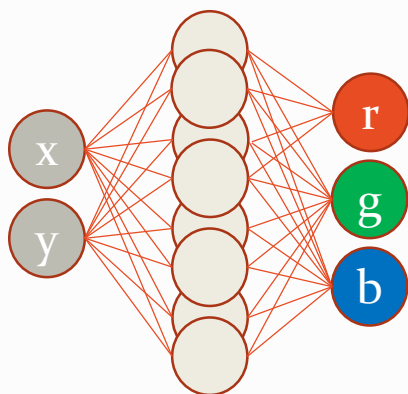
↳ Slikamo 2D v 3D: $(x,y) \mapsto (r,g,b)$

↳ Aktivacijsko funkcijo vidimo kot mehak „rez“

↳ tanh mehkejši, relu ostrejši

↳ Prva plast določa število segmentov vhodnega prostora

↳ Nič bolje kot vsota logističnih ali podobnih regresij



relu (2→8 →3)
45 parametrov

relu (2→32 →3)
165 parametrov

relu (2→128→3)
645 parametrov

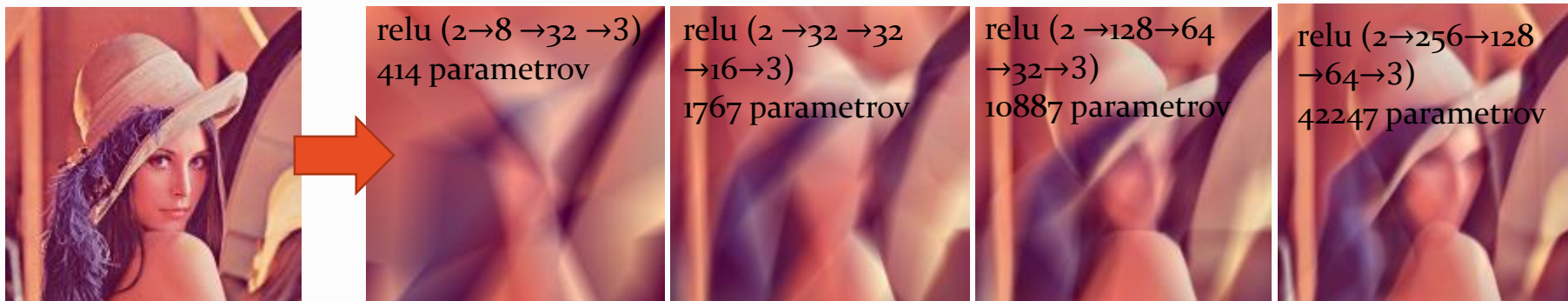
tanh (2→8 →3)
45 parametrov

tanh (2→32 →3)
165 parametrov

tanh (2→128→3)
645 parametrov

Primer

- ↳ Slikamo 2D v 3D: $(x,y) \mapsto (r,g,b)$
- ↳ Aktivacijsko funkcijo vidimo kot mehak „rez“
 - ↳ tanh mehkejši, relu ostrejši
- ↳ Višje plasti omogočajo detajle, a le znotraj značilnk, ki jih skuha prva plast (ozko grlo – ko gre podatek stran, gre stran)
- ↳ Diminishing returns – za kaj boljšega moramo upoštevati naravo vhodnih podatkov
 - ↳ Posebne arhitekture in napredni algoritmi učenja: CNN, RNN, AE,...



Compact Neural Network Solutions to Laplace's Equation in a Nanofluidic Device

Martin Magill

U. of Ontario Inst. of Tech.
martin.magill1@uoit.net

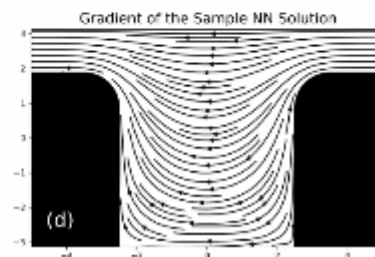
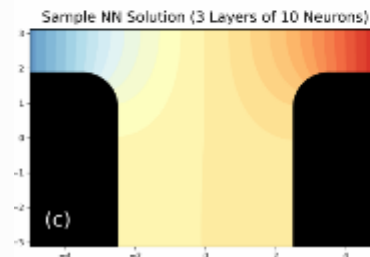
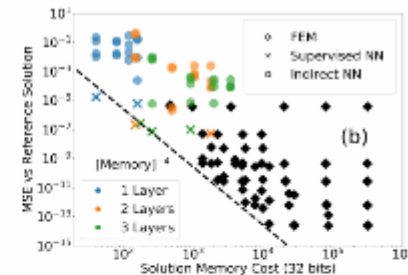
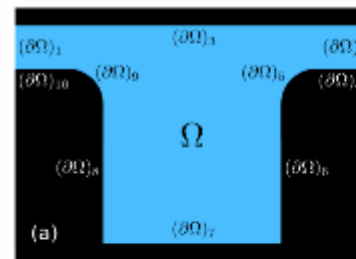
Faisal Z. Qureshi

U. of Ontario Inst. of Tech.
faisal.qureshi@uoit.ca

Hendrick W. de Haan

U. of Ontario Inst. of Tech.
hendrick.dehaan@uoit.ca

Podobno kot naša Lenna:
treniramo funkcijo $(x,y) \mapsto \text{skalar}$



Kdaj uporabiti nevronske mreže?

- ↳ Podatki slabo separabilni tudi če upoštevamo nelinearnost
 - ↳ **Hierarhična** zgradba iskanih vzorcev, visoka in slabo definirana raznolikost

Artificial neural network

- ↳ **Intuicija**: če po občutku znamo, a ne vemo kako → ANN

- ↳ Iskanje vzorcev v zapletenih podatkih, kjer ANN lahko izkoristi strukturo podatkov (CNN, npr. za slike, RNN za zvok/signale, transformerje)

Convolutional NN

Recurrent NN

- ↳ Tipično za nadzorovano učenje
- ↳ Nenadzorovano učenje: avtoenkoderji, generativni modeli, transformerji
- ↳ Vhodni seti podatkov nimajo jasno definirane statistične porazdelitve (Bayes gre skozi okno)
- ↳ Najbližje **univerzalni**
 - ↳ sposobnost učenja sklepanj in algoritmov direktno iz podatkov
 - ↳ bolj splošno orodje je tudi težje obvladljivo, če imamo specifičnega, ga uporabimo

Glej tudi: www.deeplearningbook.org

Keras



↳ Dodajanje plasti:

↳ `model.add(layers.Dense(32,` → Število nevronov
↳ `activation='relu',` → (dolžina vektorja)
↳ `kernel_initializer=...,`
↳ `kernel_regularizer=...))`

aktivacijska funkcija
,relu'
,sigmoid'
,tanh'
,softmax'
,softplus'
,selu'

regularizers.L1
regularizers.L1L2
regularizers.L2
regularizers.OrthogonalRegularizer ...

↳ Loss funkcija:

↳ `model.compile(loss=keras.losses.MeanSquaredError(),
optimizer=keras.optimizers.SGD(learning_rate=0.01))`

Probabilistic losses

- BinaryCrossentropy class
- CategoricalCrossentropy class
- SparseCategoricalCrossentropy class
- Poisson class
- binary_crossentropy function
- categorical_crossentropy function
- sparse_categorical_crossentropy function
- poisson function
- KLDivergence class
- kl_divergence function

Regression losses

- MeanSquaredError class
- MeanAbsoluteError class
- MeanAbsolutePercentageError class
- MeanSquaredLogarithmicError class
- CosineSimilarity class
- mean_squared_error function
- mean_absolute_error function
- mean_absolute_percentage_error function
- mean_squared_logarithmic_error function
- cosine_similarity function
- Huber class
- huber function
- LogCosh class
- log_cosh function

Naivni gradientni spust:

```
tf.keras.optimizers.SGD(  
    learning_rate=0.01, momentum=0.0, nesterov=False, name="SGD", **kwargs  
)
```

Običajno se kliče bolj napredne: ,adam', ... (glej dokumentacijo)

↳ Sledi `model.fit` (to poznate)

↳ `model.predict` za izračun rezultata na novem vhodu

Povezave

↳ Tutoriali in lekcije:

↳ A. Geron – Hands-On ML with Scikit-Learn & TF

↳ www.deeplearningbook.org

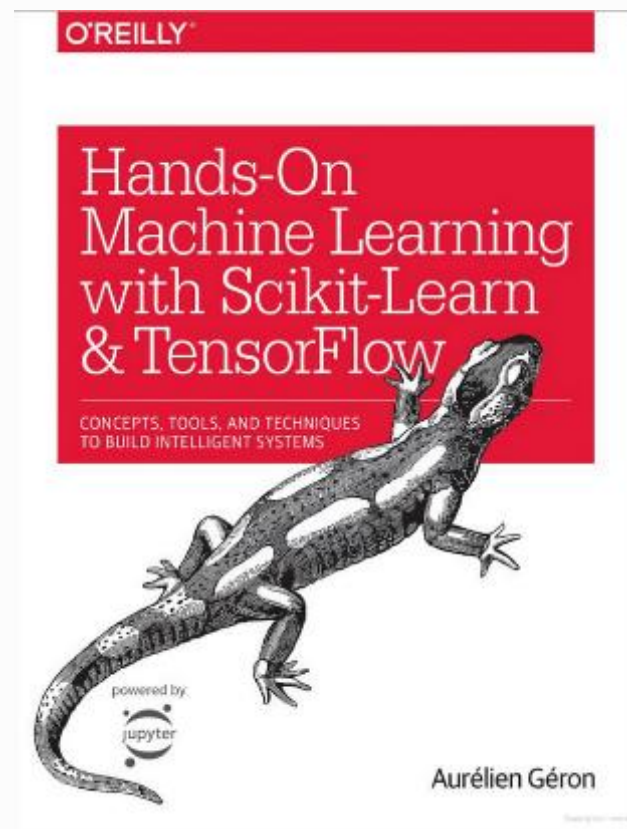
↳ www.machinelearningmastery.com/start-here

↳ Dokumentacija:

↳ keras.io

↳ tensorflow.org

↳ scikit-learn.org



Fizične realizacije NN paradigme

- ↳ Sklopljeni sistemi: flow networks, elastic networks
 - ↳ Izziv je trening: prilagoditev prepustnosti?

PHYSICAL REVIEW X **11**, 021045 (2021)

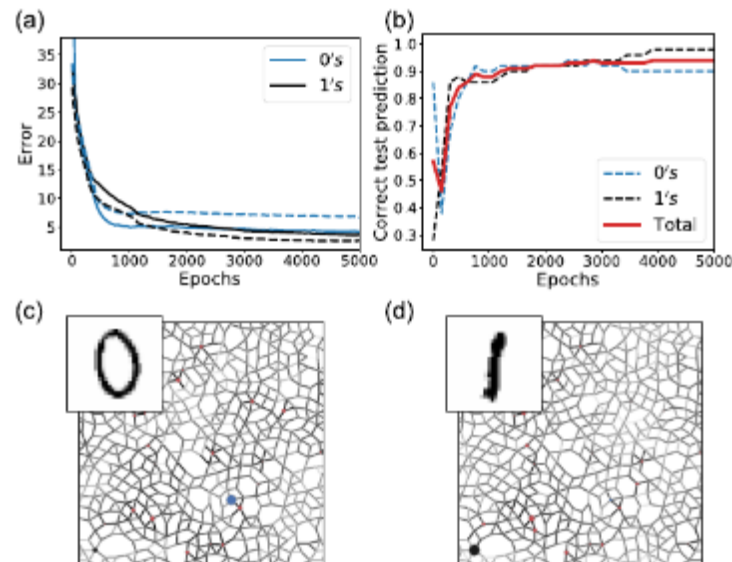
Supervised Learning in Physical Networks: From Machine Learning to Learning Machines

Menachem Stern¹, Daniel Hexner², Jason W. Rocks³ and Andrea J. Liu¹

¹*Department of Physics and Astronomy, University of Pennsylvania,
Philadelphia, Pennsylvania 19104, USA*

²*Mechanical Engineering, Technion, Haifa, 32000, Israel*

³*Boston University, Boston, Massachusetts 02139, USA*



Fizične realizacije NN paradigme



FMF

UNIVERZA
V LJUBLJANI

Fakulteta za matematiko
in fiziko

↳ Optične nevronske mreže

↳ Faza namesto amplituda: interferenčni pojavi

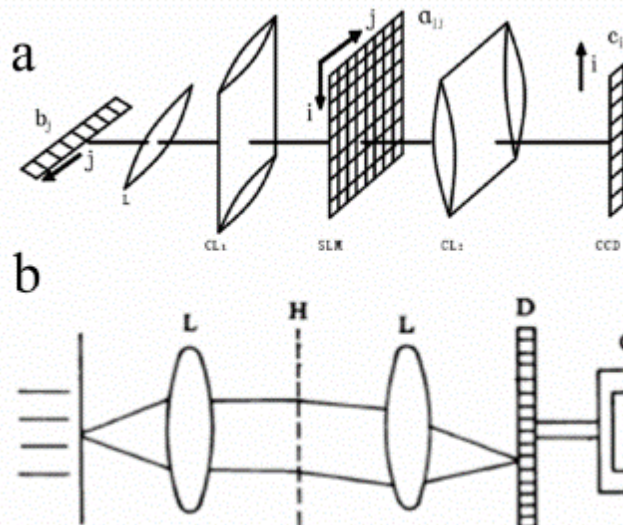
↳ Trening softverski, evaluacija pa optična: paralelno, instanten rezultat

Science

REPORTS

Cite as: X. Lin *et al.*, *Science*
10.1126/science.aat8084 (2018).

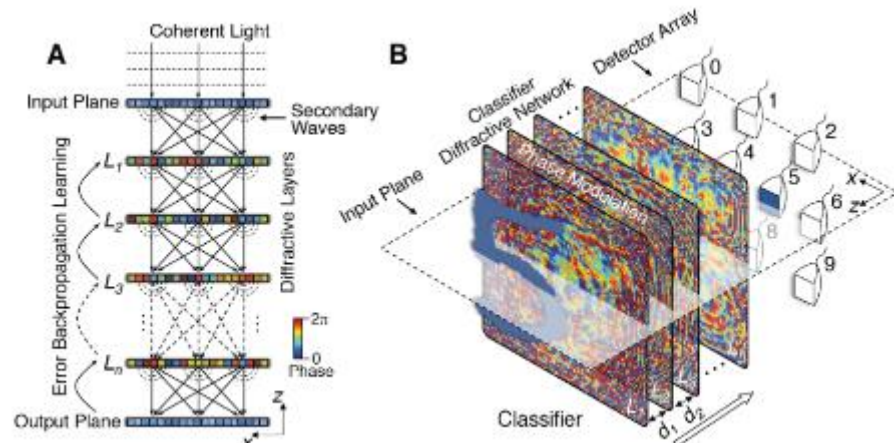
All-optical machine learning using diffractive deep neural networks



^{1,2,3}, Yi Luo^{1,2,3}, Mon
CA, 90095, USA. ³Bioe
Angeles, CA, 90095, US

ecute advanced
earning by dem
plement variou
it work collecti
n digits and fas
cal deep learni

er-based neural networks can implement, and will
detection and object classification, also enabling
form unique tasks using D²NNs.



Model nevronske mreže

↳ Zaenkrat „feed-forward“

↳ Podatki z leve proti desni po plasteh, nič rekurzije

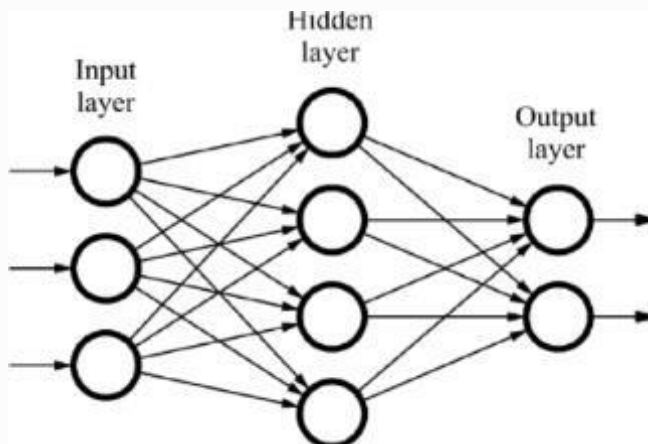
↳ `keras.Sequential`

↳ Ponovimo:

$$\mathbf{y} = f \left(W^{(3)} f \left(W^{(2)} f \left(W^{(1)} \mathbf{x} \right) \right) \right)$$

↳ Iščemo uteži W

↳ Minimiziramo cenovno funkcijo (**loss function / fitness function**) $J_{W(j)}(\mathbf{y}, \mathbf{t})$



Gradientni spust

↳ Gradient nam pove popravek uteži:

$$W_{ij}^{(k)} \mapsto W_{ij}^{(k)} - \eta \frac{1}{n} \sum_{\text{minibatch}} \frac{\partial J}{\partial W_{ij}^{(k)}}$$

↳ Povprečimo po minibatchih

↳ Podatki ne smejo biti sortirani

↳ Zaidemo v lokalne ekstreme, overfitamo del podatkov

↳ Poseben primer: coordinate descent

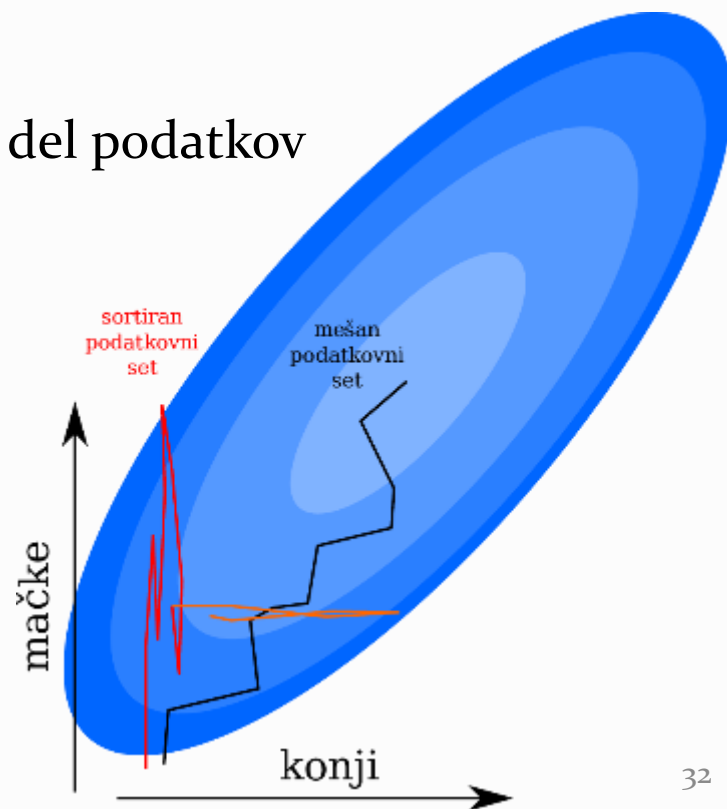
↳ treniranje vsake dimenzije posebej

↳ uporabno pri L1 regularizaciji

↳ Izračun gradienta je glavni korak!

↳ Odvisen od arhitekture mreže

↳ Propagacija skozi mrežo



Vzvratna propagacija napake

↳ Optimiziramo kompozitum: gradient po verižnem pravilu

$$y = f \left(W^{(3)} f \left(W^{(2)} f(W^{(1)} x) \right) \right)$$

$\tilde{y}^{(2)}$ izhod prejšnje plasti

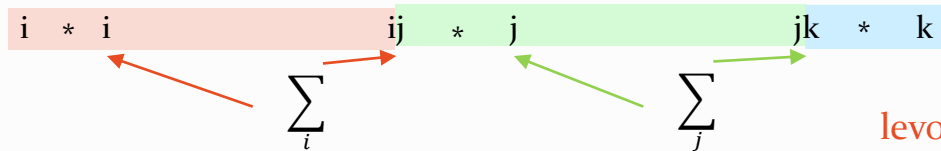
$y^{(2)}$ po aktivaciji

$$\frac{\partial J}{\partial W^{(3)}} = \underline{(\nabla J)(f'(W^{(3)} y^{(2)}))} \otimes y^{(2)}$$

$$\frac{\partial J}{\partial W^{(2)}} = \underline{((\nabla J)(f'(W^{(3)} y^{(2)})) W^{(3)} (f'(W^{(2)} y^{(1)})))} \otimes y^{(1)}$$

$$\frac{\partial J}{\partial W^{(1)}} = \underline{((\nabla J)(f'(W^{(3)} y^{(2)})) W^{(3)} (f'(W^{(2)} y^{(1)})) W^{(2)} (f'(W^{(1)} x)))} \otimes x$$

33



ni čisto sumacijsko pravilo,
levo od funkcije množimo po komponentah

↳ „Stochastic gradient descent“: običajno minimiziramo v manjših paketih (minibatches)

32-1024, po pameti glede na spomin in paralelizacijo

↳ Načrtovanje hitrosti učenja (learning rate scheduling)

↳ Hitrejša minimizacijske sheme: momentum-based (pospešek)

Težave z gradienti

↳ Propagacija = iteracija: norma se v povprečju množi s konstanto

↳ $q > 1$: gradienti eksplodirajo, popravki v prvih plasteh preveliki

↳ $q < 1$: gradienti izginejo, prve plasti se ne učijo

↳ Glavni problem večine NN metod!

↳ Prevelika globina je težavna

↳ Strategije:

↳ pametna inicializacija uteži

↳ menjava aktivacijske funkcije

↳ ReLU in sorodne namesto tanh (da ni saturacije)

↳ regularizacija

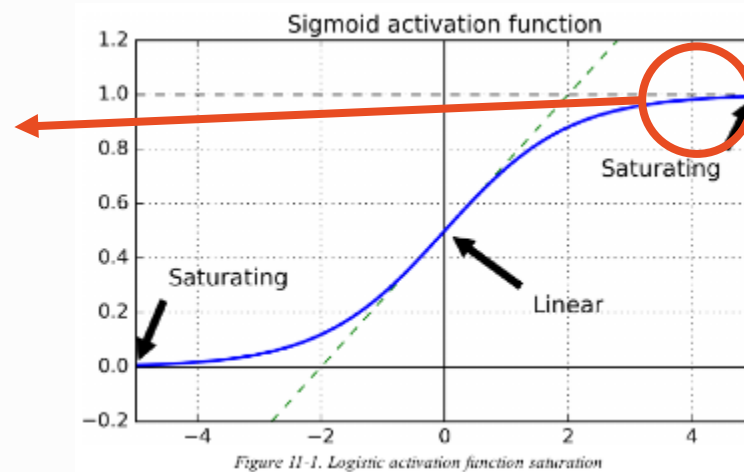
↳ treniranje dela uteži, postopno dodajanje plasti (layer-wise pretraining)

↳ Batch normalization: normalizacija vmesnih rezultatov

↳ Gradient clipping

Glej Gerona za diskusijo in primere

↳ renormalizacija uteži Glej Gerona, `kernel_constraint=keras.constraints.max_norm(...)`



Napredne metode minimizacije

relativno napredne:
zelo simplistično v primerjavi
z ODE integratorji – RK,
prediktor-korektor,
Richardsonova ekstrapolacija

↳ Še vedno je gradient glavni del izračuna

↳ Nadgradnja za gradient descent:

↳ *momentum-based minimization* (Nesterov)

↳ pospešek + hitrost: enačba drugega reda (fizikom zelo domače)

↳ $m_i \rightarrow \beta m_i - \eta g_i$ → $\mathbf{g} = \nabla J$

↳ $\theta_i \rightarrow \theta_i + m_i$ → momentum parameter: dušenje

↳ AdaGrad: → popravek (korak) metode

↳ Normalizacija gradientov z drsečim povprečjem

$$m_i \rightarrow -\eta \frac{g_i}{\sqrt{\sum_{t'} g_{i,t'}^2}}$$

↳ Adam:

↳ Geometrijska relaksacija gradientov in varianc (posplošitev zgornjega)

$$V_i \rightarrow \gamma V_i + (1 - \gamma) \sum_{t'} g_{i,t'}^2$$

$$m_i \rightarrow \beta m_i + (1 - \beta) g_i$$

$$\theta_i \rightarrow \theta_i - \eta \frac{\hat{m}_i}{\sqrt{\hat{V}_i}}$$

počasen vklop pospeška

$$\hat{m}_i = \frac{m_i}{1 - \beta^t} \quad \hat{V}_i = \frac{V_i}{1 - \gamma^t}$$

primerjaj: Čebišev pospešek

↳ Bolj pametni adaptivni minimizatorji

↳ Samo izberemo iz seznama

↳ `keras.optimizers.*`

Napredne metode minimizacije

↳ Poleg izbire minimizatorja tudi adaptivno spreminjanje koraka

↳ `keras.optimizers.schedules.*`

↳ `learning_rate=schedule` ← namesto surove številke

↳ Alternativa gradientni metodi:

↳ <https://arxiv.org/pdf/1605.02026.pdf>

$$\begin{aligned} & \underset{\{W_l\}, \{a_l\}, \{z_l\}}{\text{minimize}} && \ell(z_L, y) \\ & \text{subject to} && z_l = W_l a_{l-1}, \text{ for } l = 1, 2, \dots, L \\ & && a_l = h_l(z_l), \text{ for } l = 1, 2, \dots, L-1. \end{aligned}$$

$$\begin{aligned} & \underset{\{W_l\}, \{a_l\}, \{z_l\}}{\text{minimize}} && \ell(z_L, y) \\ & && + \langle z_L, \lambda \rangle + \beta_L \|z_L - W_L a_{L-1}\|^2 \\ & && + \sum_{l=1}^{L-1} [\gamma_l \|a_l - h_l(z_l)\|^2 + \beta_l \|z_l - W_l a_{l-1}\|^2] \end{aligned}$$

↳ Povsem drugačen pristop:

Učenje z nagrajevanjem (reinforcement learning)

↳ Genetski algoritmi

↳ Uporabno, če nimamo odvedljive cenovne funkcije (npr. rezultat neke kompleksne simulacije ali celo eksperimenta)

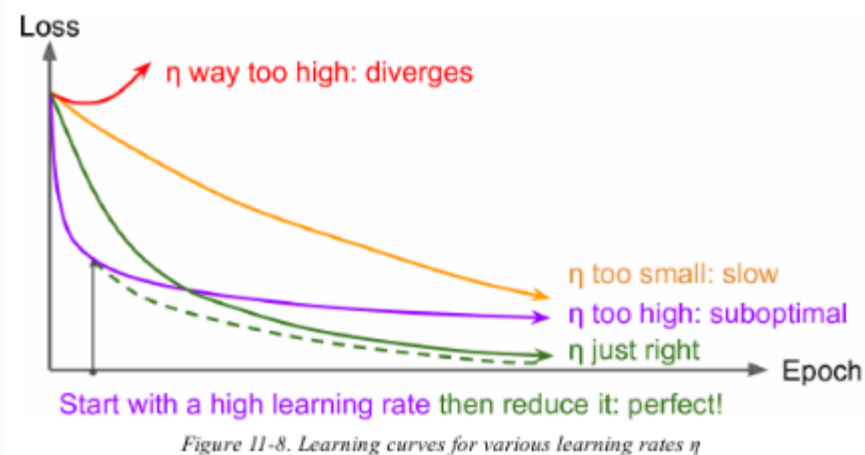
↳ Izmenjave nevronov med predniki (crossover) + mutacije

Avtomobilčki: <https://www.youtube.com/watch?v=yH21FdWP-oo>

Kače: <https://www.youtube.com/watch?v=BBLJFYr7zB8>

Načrtovanje učenja

- ↳ Naivno: $\eta = 0.01$
- ↳ Hitrost učenja (**learning rate**): prevelika – lokalni ekstremi, premajhna – počasna konvergenca
- ↳ Iščemo globalni minimum: sprotno spreminjanje?
 - ↳ Težave s sedli
 - ↳ Najprej veliki koraki, potem manjši?
 - ↳ Primerjaj: **simulirano ohlajanje** (**simulated annealing**)
 - ↳ Adaptivno (kot Runge-Kutta, itd?)
 - ↳ Uporaba drugega odvoda, Hessian
 - ↳ Primerjaj: **Levenberg-Marquardt**
 - ↳ Ciklanje skozi komponente ciljnega vektorja
 - ↳ **Dynamical loss function**
 - ↳ <https://arxiv.org/abs/2102.03793>



↳ Popularne opcije:

↳ `tf.keras.optimizers.schedules.*`

↳ Power/exponential scheduling

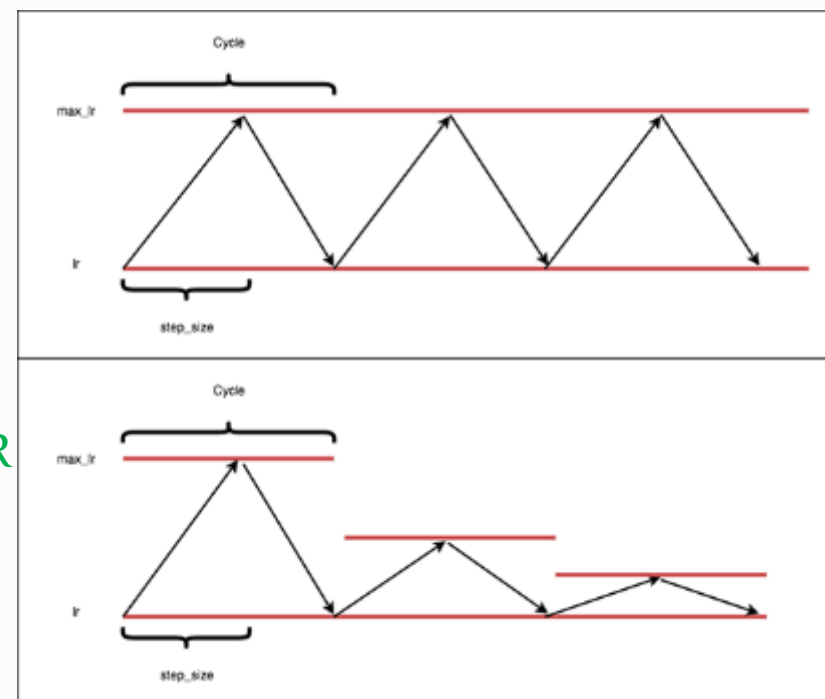
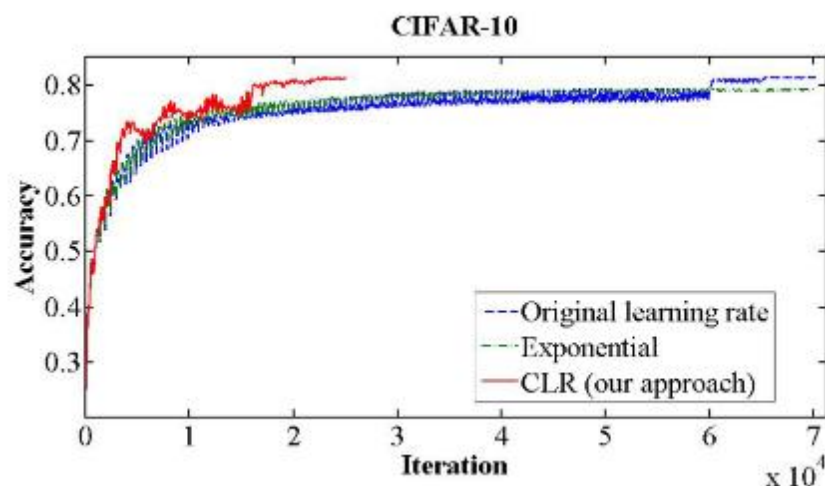
↳ Performance scheduling

↳ Glede na validacijsko napako

↳ Cyclical learning rate (CLR):

↳ Izmenjava kratkih in dolgih potez

↳ <https://github.com/bckenstler/CLR>



Težave z overfittingom in konvergenco

- ↳ Želimo, da algoritem zna **posplošit** podatke in da skonvergira
- ↳ Regularizacija uteži L1, L2 (**primerjaj: Ridge, Lasso, Elastic Net**)
 - ↳ `keras.regularizers.l1_l2(l1=...,l2=...)`
- ↳ Dropout (Naključno ugašamo nevrone, prisilimo, da se znajde)
 - ↳ `keras.layers.Dropout`
- ↳ Preveč nevronov vmesnih plasteh: neizkoriščeni nevroni, ki ostanejo naključni, uteži naslednje plasti pa nizke
 - ↳ Smiselno je gledat uteži in aktivacije nevronov med validacijo
 - ↳ Ortogonalizacija uteži (pri inicializaciji, ali kot regularizacijski korak)
 - ↳ Ekvivalentno vezanemu ekstremu $\lambda ||W^T W - I||^2$
- ↳ Premalo nevronov: ozko grlo informacije – izguba dela informacije
 - ↳ lahko to želimo: redukcija dimenzij, kompresija, posploševanje
- ↳ Izbira ustrezne razdelitve za cross-validation
- ↳ Pazimo na „data leakage“ – korelirani vzorci v testnem in validacijskem naboru
 - ↳ Npr.: podvojeni podatki – isti vzorec za test in validacijo
 - ↳ Npr.: časovna vrsta – naslednji vzorci so pod vplivom prejšnjih

Priprava učnega seta vhodnih podatkov

↳ Training – Validation – Test set

- ↳ Nevarnost: „leakage“ – korelirani učni in validacijski podatki
- ↳ Naključno pomešani podatki (**problem: časovna zaporedja**)
- ↳ **Validation:** izmera natančnosti **med** treningom (indikator konvergence)
- ↳ Cross-validation (glej prejšnje slajde)
- ↳ **Test:** končna evaluacija (povsem ločeni podatki)

↳ Data augmentation (**Najpomembnejši del!**)

- ↳ Generiramo sintetične vhode, pridobimo večji set
- ↳ Invarianca na transformacije:
- ↳ Šum, translacije in rotacije, deformacije (pri slikah), skaliranje, maskiranje
- ↳ Pogosto je **simulacija** edini način
 - ↳ premalo realnih vzorcev oziroma fizikalno neizvedljivi parametri

↳ Dovolj raznolikosti, da je posplošitev mogoča

↳ Balansiranje deležev različnih razredov

- ↳ *Primer: 90% kontrolnih, 10% rakavih slik*
- ↳ *Strategija: utežena loss funkcija (**weighted cross entropy**)*
- ↳ *Pazljivo pri izbiri validacijskega in testnega nabora*
 - ↳ *Nevarnost, da ni nobene rakaste slike v testnem naboru*

Optimizacija hiperparametrov

↳ Veliko diskretnih parametrov nevronske mreže:

- | | | |
|---------------------------------|---|---|
| ↳ Tip aktivacijske funkcije | } | arhitektura mreže (vpliva tudi na izvedenost) |
| ↳ Število plasti | | |
| ↳ Število nevronov po plasteh | | |
| ↳ Število epoh | } | strategija treniranja (vpliva le med treningom) |
| ↳ Hitrost učenja / načrt učenja | | |
| ↳ Optimizator | | |

↳ Grid search – brute force, vse kombinacije poskusimo

- ↳ Tukaj primerjamo uspešnost (accuracy, loss function, sensitivity,...)
 - ↳ Na **testnem** setu podatkov
- ↳ Slabost: dolgo traja!
- ↳ Možnost: vsak hiperparameter ločeno (požrešna metoda)

↳ Genetski algoritem

- ↳ Populacija različnih mrež
- ↳ Obdržimo najboljše
- ↳ Naključne kombinacije najboljših + mutacije
- ↳ Iteracija

↳ Odziv na nevzorčen del vhodnega prostora je nedefiniran

↳ Klasifikatorju številke ne dajamo črk

↳ Problem: anomaly detection - k temu se še vrnemo

↳ Smiselno si je ogledat napake

↳ Je napaka „odpustljiva“ (glej desno), ali nakazuje na slabo trenirano mrežo?



↳ Transfer learning:

↳ Uporaba že naučenih plasti (`model.layers[n].trainable = False`)

↳ Ne izumljamo tople vode: vzamemo že trenirano mrežo!

↳ Prve plasti so lahko enake, samo klasifikator zamenjamo

↳ ImageNet, CIFAR

↳ Primer: prepoznavanje drugačnih signalov se lahko opira na iste lokalne značilnosti (zaznavanje vrhov, stopnic, valov,...)

↳ „bazo“ vzamemo od drugega problema, koeficiente interpretiramo na novo

Fizikalno informirane nevronske mreže

Physically-informed NN



FMF

UNIVERZA
LJUBLJANA

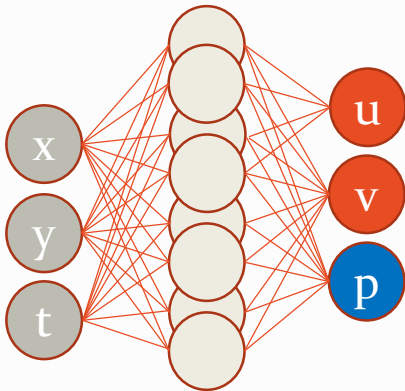
Fakulteta za matematiko
in fiziko

↳ Rešujemo parcialne diferencialne enačbe

↳ Hidrodinamika, difuzijska enačba,...

↳ Na vrat na nos:

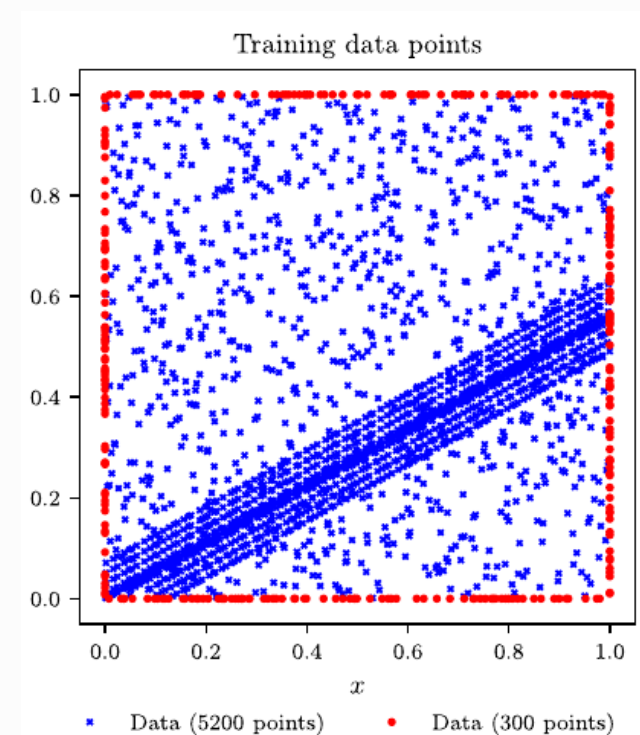
↳ Iščemo funkcijo $f(x,y,t)$: vzamemo kar nevronske črno škatlo



↳ **Treniramo na:** vzorcih na čim bolj smiselno porazdeljenih točkah v prostoru in času

↳ **Začetni in robni pogoji** podobno

↳ **Prednost:** lahko meritve na senzorjih!



↳ Kako zagotovimo, da je izpolnjena diferencialna enačba?

↳ Minimiziramo **residual diferencialne enačbe**

$$\rho \frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} = -\nabla p + \eta \nabla^2 \mathbf{v} \longrightarrow \left(\rho \frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} + \nabla p - \eta \nabla^2 \mathbf{v} \right)^2$$

↳ Bolj korektno: **variacijski princip** – minimiziramo akcijo (Lagrangian)

↳ Odvode znamo računat s propagacijo napake – „zastonj“ funkcionalnost

↳ Cenovna funkcija:

↳ Kar utežena vsota izgube podatkov in izgube residuala (in ostalih prispevkov ki jih hočemo penalizirat, vključno z vsemi vezmi, **variacijski račun prenese vse**)

$$Loss = Loss_{Data} + Loss_F,$$

↳ Mrežo učimo spoštovanja fizikalnih zakonov

↳ Dobimo kar „celo“ rešitev, funkcijo prostora in časa, ne gre za propagacijo

- ↳ „Forward“ problem: iščemo funkcijo, ki jo želimo iz vrednotiti
 - ↳ Lahko gre za rekonstrukcijo iz meritev
 - ↳ Lahko treniramo na simuliranih podatkih
- ↳ Inverzni problem: rekonstrukcija iz rezultatov (iz optične slike, gradientov, itd...)
- ↳ Iskanje parametrov: iščemo parametre v diferencialni enačbi
 - ↳ Viskoznost, toplotna prevodnost, gostota,...
 - ↳ Nekaj podobnega ste počeli pri iskanju elastičnih konstant iz transmisije

Physics-informed neural networks for high-speed flows

Zhiping Mao, Ameya D. Jagtap, George Em Karniadakis*

Division of Applied Mathematics, 182 George Street, Brown University, Providence, RI 02912, USA

Received 25 May 2019; received in revised form 16 December 2019; accepted 17 December 2019

Available online xxx

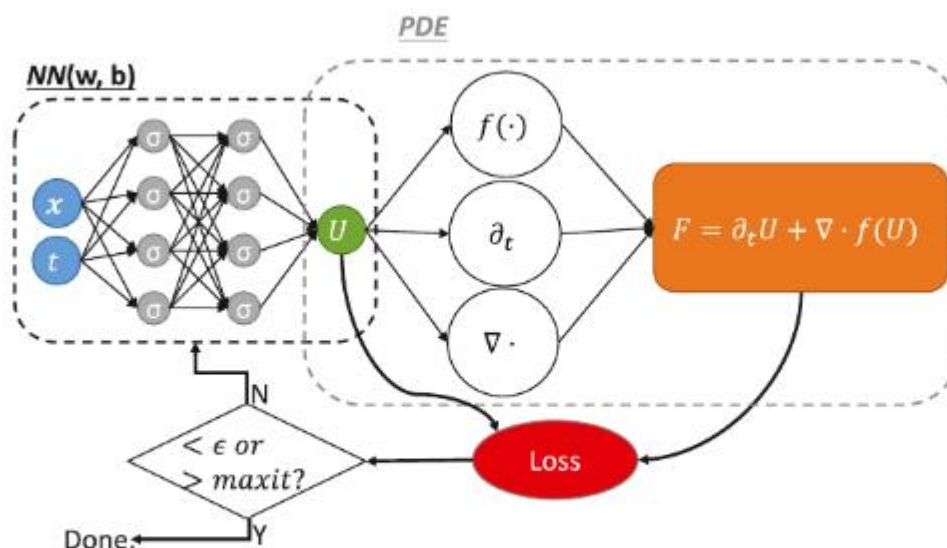


Fig. 1. Schematic of PINN for the Euler equations. The left NN is the uninformed while the right one induced by the conservation law is the informed one. The two NNs share hyper-parameters and they both contribute to the loss function.



UNIVERZA
V LJUBLJANI

FMF

Fakulteta za matematiko
in fiziko

Physics-Informed Neural Networks for Heat Transfer Problems

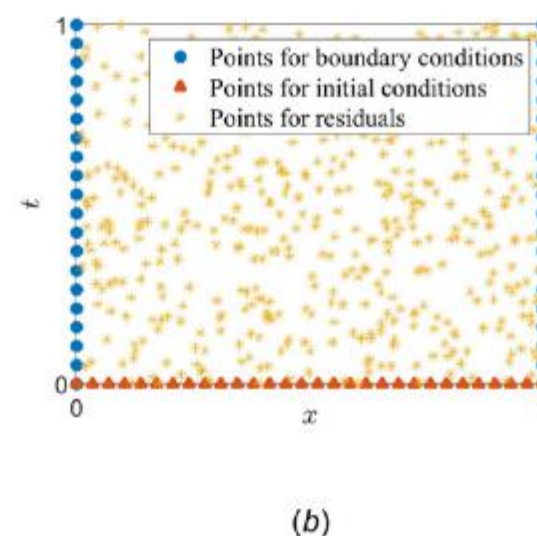
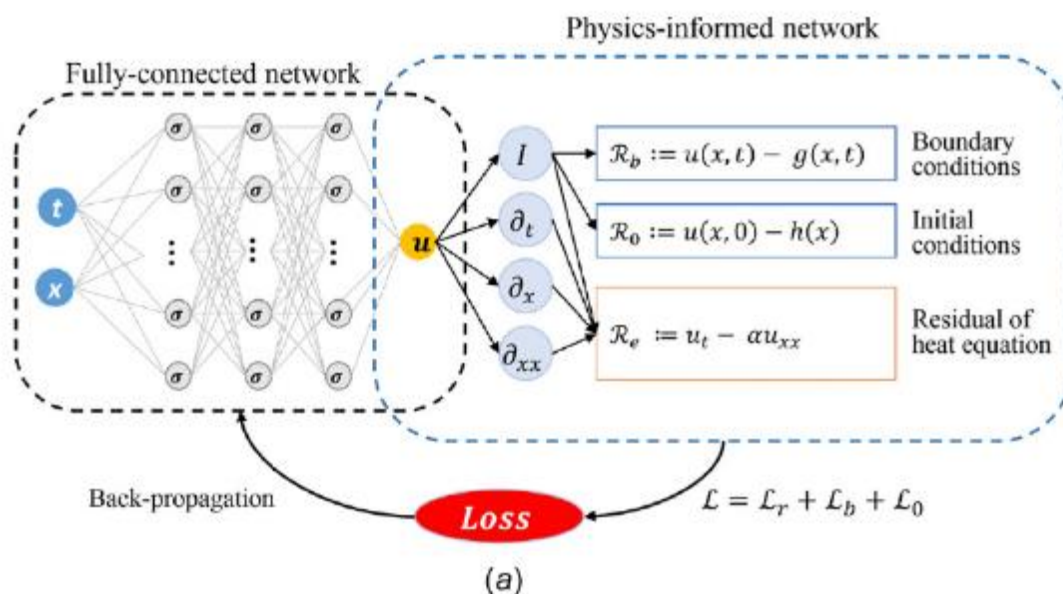


Fig. 1 Overview of physics-informed neural networks (PINNs). (a) Schematic of PINN framework. A fully connected neural network is used to approximate the solution $u(x, t)$, which is then applied to construct the residual loss $\mathcal{L}_r$, boundary conditions loss $\mathcal{L}_b$ and initial conditions loss $\mathcal{L}_0$. The derivatives of u are computed by automatic differentiation in TensorFlow [26]. The parameters of the fully connected network are trained using gradient-descent methods based on the back-propagation of the loss function. (b) Schematic of point selection for PINN in the computational domain. The points with different colors correspond to different terms in the loss function. Note that the residual points for computing the residual loss can be randomly selected in the space-time domain.

VPINNs: VARIATIONAL PHYSICS-INFORMED NEURAL NETWORKS FOR SOLVING PARTIAL DIFFERENTIAL EQUATIONS *

EHSAN KHARAZMI [†], ZHONGQIANG ZHANG [‡], GEORGE EM KARNIADAKIS^{§¶}

VPINN: Variational Physics – Informed Neural Network

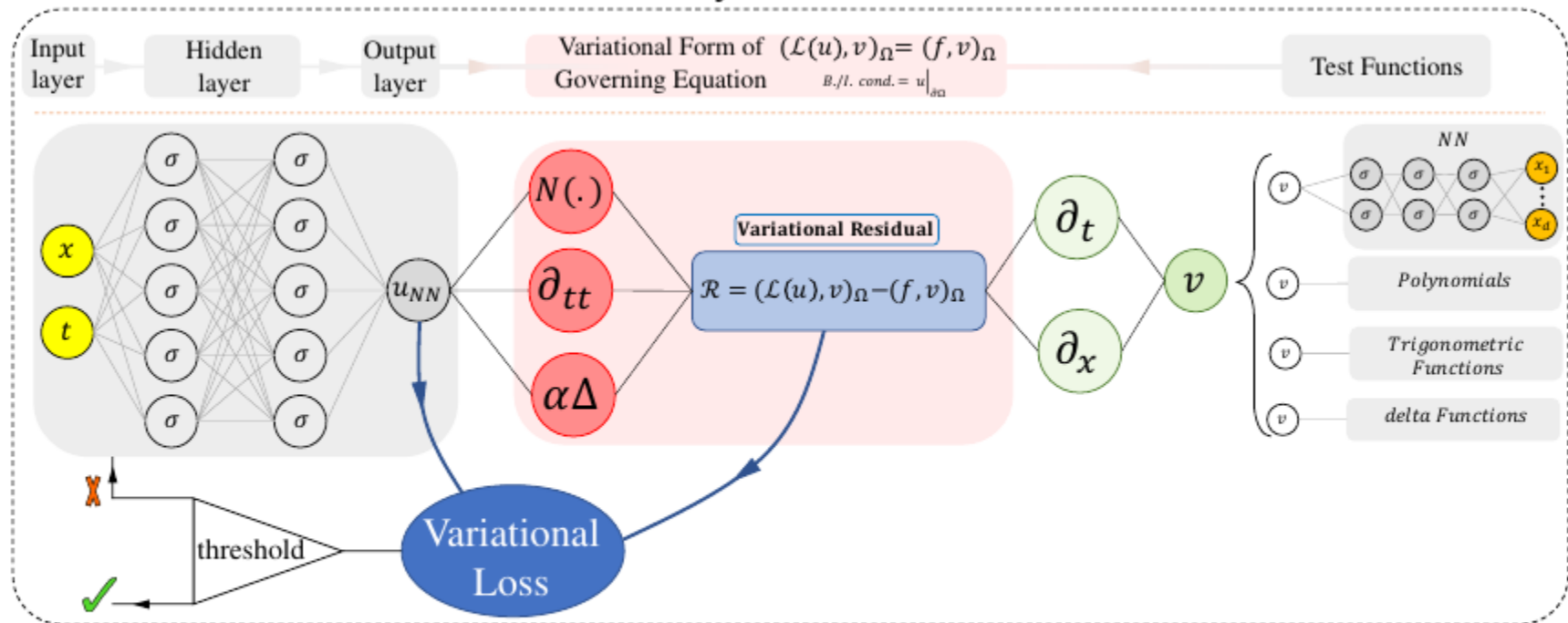
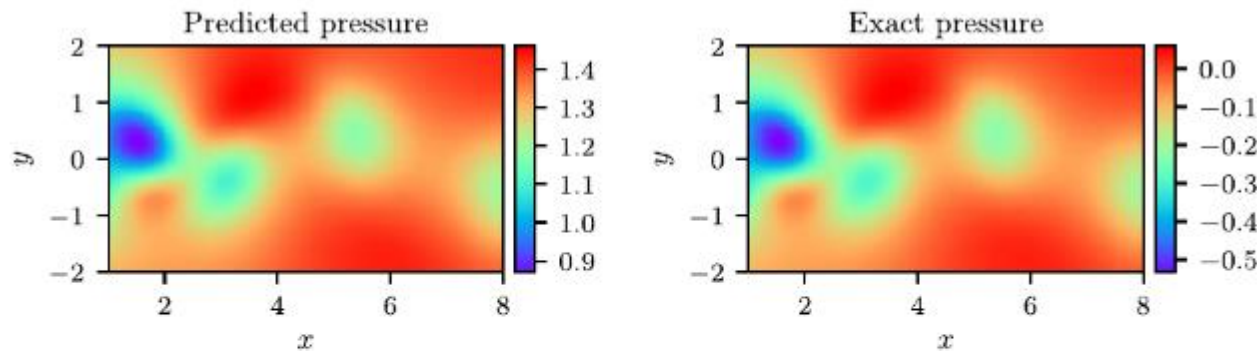


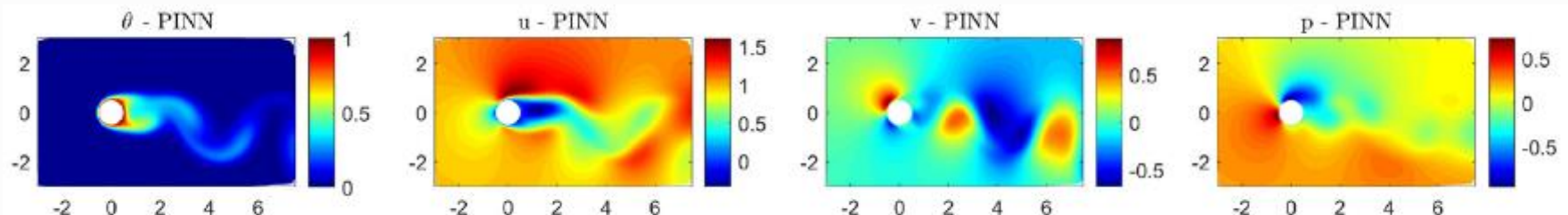
Fig. 2: Schematic of VPINN in a Petrov-Galerkin formulation. Trial functions belong to the space of NN and test functions can be chosen from a separate NN or other function spaces such as polynomials and trigonometric functions. Red color represents the differential operators on trial space. Green color represents the test functions and their derivatives. Blue color represents the variational residuals $\mathcal{R}$.

VPINNs: VARIATIONAL PHYSICS-INFORMED NEURAL NETWORKS FOR SOLVING PARTIAL DIFFERENTIAL EQUATIONS *

EHSAN KHARAZMI [†], ZHONGQIANG ZHANG [‡], GEORGE EM KARNIADAKIS^{§¶}



Correct PDE	$u_t + (uu_x + vv_y) = -p_x + 0.01(u_{xx} + u_{yy})$ $v_t + (uv_x + vv_y) = -p_y + 0.01(v_{xx} + v_{yy})$
Identified PDE (clean data)	$u_t + 0.999(uu_x + vv_y) = -p_x + 0.01047(u_{xx} + u_{yy})$ $v_t + 0.999(uv_x + vv_y) = -p_y + 0.01047(v_{xx} + v_{yy})$
Identified PDE (1% noise)	$u_t + 0.998(uu_x + vv_y) = -p_x + 0.01057(u_{xx} + u_{yy})$ $v_t + 0.998(uv_x + vv_y) = -p_y + 0.01057(v_{xx} + v_{yy})$



Hebbova teorija

↳ Alternativen pogled na nevronske mreže

↳ „*Neurons that fire together, wire together*“

↳ Ojačamo uteži, ki vodijo v ojačanje že vzpostavljenih povezav

↳ $\Delta \mathbf{w}_i = \eta y_i \mathbf{x} - \eta y_i \mathbf{w}$ weight decay (regularizacija)

↓
uteži za konkretni nevron

→ vhod nevrona
→ izhod nevrona po aktivacijski funkciji (post-synaptic activation); $y > 0$

↳ Uteži si „zapomnijo“ najbolj tipične, najpogostejše vhode

↳ Nenadzorovano učenje, gradnja **asociacij**, prepoznavna **vzorcev**

↳ Izpeljanka

↳ **Meta-učenje**: učenje pravil, po katerih ojačujemo povezave

↳ Uteži niso več statične, ampak se prilagajajo inputu

↳ „Hebbian plasticity“

↳ <https://arxiv.org/pdf/2007.02686.pdf>

Uporaba nevronske mreže

- ↳ Klasifikacija in kategorizacija (e.g.: klasifikacija slik galaksij, prepoznavanje materialov po spektru, klasifikacija trkov v pospeševalnikih)
- ↳ Regresija (e.g.: vrednosti zemljišč, fitanje funkcij, iskanje elastičnih konstant)
- ↳ Predikcija/ekstrapolacija (e.g.: napoved vremena, reševanje PDE – Navier-Stokes, FlowNET)
- ↳ Reprezentacija (e.g.: fingerprinting - vektor generiran iz slike obraza)
- ↳ Kompresija (e.g.: čiščenje slik, redukcija dimenzij, clustering, feature extraction)
- ↳ Inverzni problemi (e.g.: rekonstrukcija CT slik, optimalna mehanska konstrukcija, iskanje tekočerkristalne konfiguracije za željen optični odziv)
- ↳ Generiranje (e.g.: GAN, deep fake, krpanje lukenj v podatkih, DALL-E, stable diffusion, foto filtri – kamere v pametnih telefonih)
- ↳ Časovna zaporedja (e.g.: prepoznavanje zvoka in govora, detekcija dogodkov v signalih)
- ↳ Kontrolni sistemi (e.g.: real time krmilniki robotov/dronov/nadzornih sistemov)
 - ↳ https://www.youtube.com/watch?v=Cu8YqGj_j1I

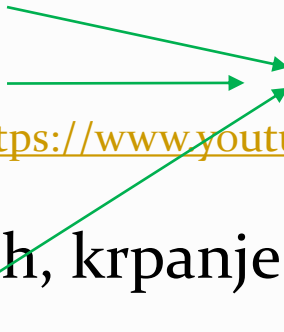
Več primerov uporabe NN

↳ Reševanje diferencialnih enačb

- ↳ Laplace, Poisson (Kolluru, arXiv: 1909.11082v1)
- ↳ Navier-Stokes

↳ Hitra približna simulacija kompleksnih sistemov

- ↳ Simulacije tkanin
- ↳ Simulacije tekočin
- ↳ Simulacija fizike (<https://www.youtube.com/watch?v=VMCYRCCqR5Q> in asociiran članek)



CGI

↳ Dvig detajlov v slikah, krpanje manjkajočih podatkov

- ↳ Nvidia OptiX

↳ Pametno filtriranje zvoka

↳ Analitični nadomestek za težje izračunljivo funkcijo

↳ Avtonomna vožnja, računalniški vid

↳ Za zabavo in dodatne ideje:

- ↳ <https://www.youtube.com/watch?v=i6wo7KwYkGw>
- ↳ Youtube: two minute papers

Machine learning phases of matter

Juan Carrasquilla  & Roger G. Melko

Nature Physics **13**, 431–434(2017) | [Cite this article](#)

19k Accesses | 430 Citations | 155 Altmetric | [Metrics](#)



FMF

UNIVERZA
V LJUBLJANI

Fakulteta za matematiko
in fiziko

particles, reminiscent of the ‘curse of dimensionality’ commonly encountered in machine learning². Despite this curse, the machine learning community has developed techniques with remarkable abilities to recognize, classify, and characterize complex sets of data. [Here](#), we show that modern machine learning architectures, such as fully connected and convolutional neural networks³, can identify phases and phase transitions in a variety of condensed-matter Hamiltonians. Readily programmable through modern software libraries^{4,5}, neural networks can be trained to detect multiple types of order parameter, as

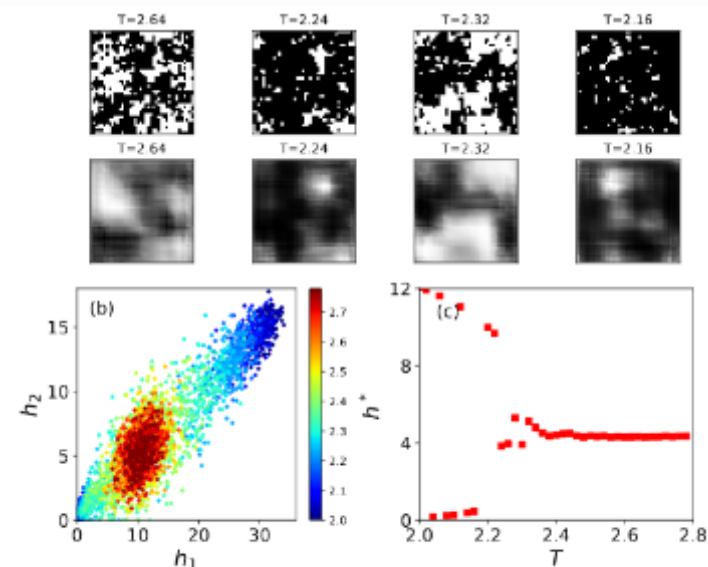


FIG. 13: Ising model autoencoder results. $J = 1.0$ is fixed and the original lattice size is $N = 40 \times 40$. (a) Spin configuration (above) and reconstructions (below) at the indicated temperatures using 200 hidden neurons. (b) Encoding of the raw Ising configurations $\{\mathbf{S}_i\}$ onto the plane of the two hidden neuron activations (h_1, h_2). Color bar indicates the temperature of the sample in units of J . Each temperature has 100 scatter points. (c) Encoding of the raw Ising configurations $\{\mathbf{S}_i\}$ using a single hidden neuron activation (h^*) as a function of temperature. Each data point is averaged over $n = 10000$ samples of the same temperature.

Discovering Phases, Phase Transitions and Crossovers through Unsupervised Machine Learning: A critical examination

Wenjian Hu^{1,2}, Rajiv R.P. Singh¹, and Richard T. Scalettar¹

¹Department of Physics, University of California Davis, 95616 CA USA

²Department of Computer Science, University of California Davis, 95616 CA USA

Fast automated analysis of strong gravitational lenses with convolutional neural networks



FMF

ŹZA
ANI

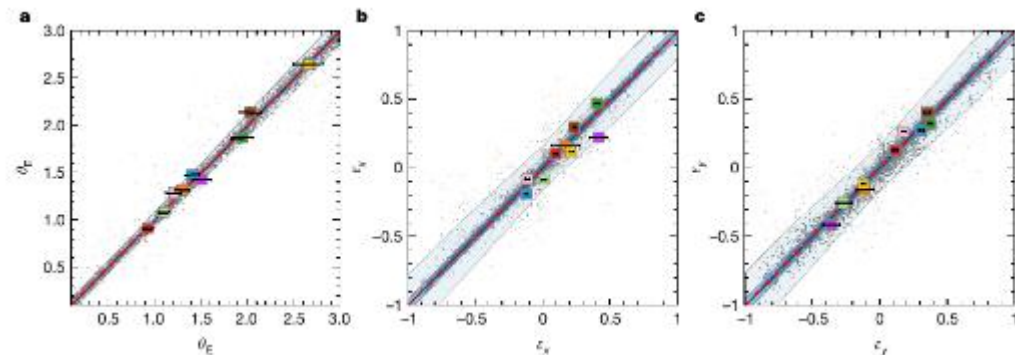
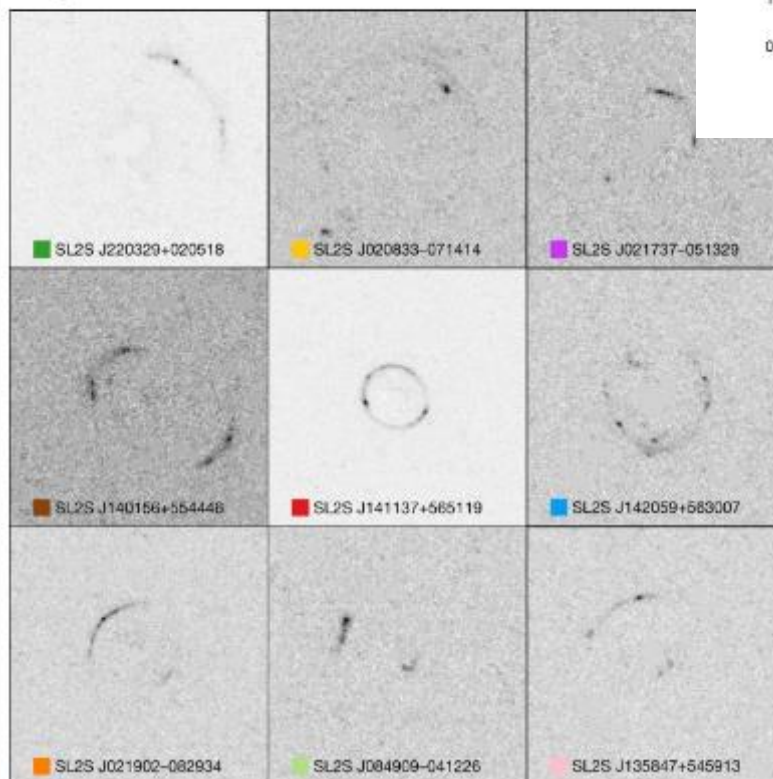
Fakulteta za matematiko
in fiziko

Yashar D. Hezaveh , Laurence Perreault Levasseur  & Philip J. Marshall

Nature **548**, 555–557(2017) | [Cite this article](#)

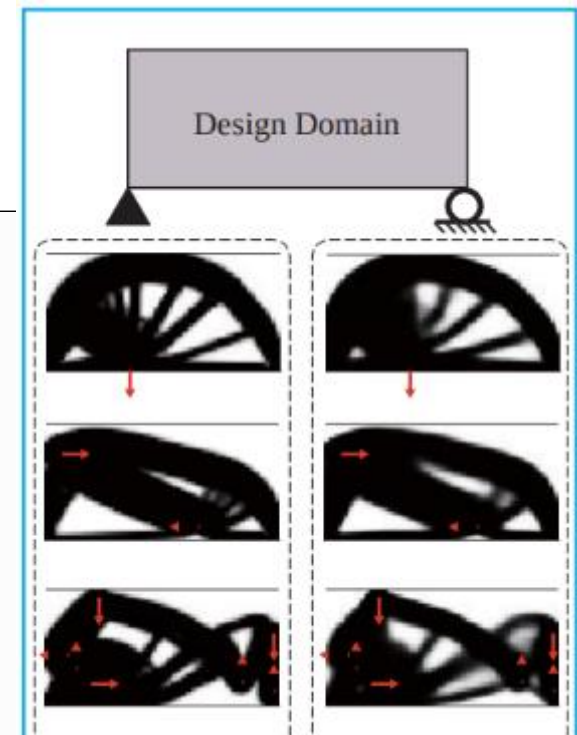
2159 Accesses | 66 Citations | 347 Altmetric | [Metric](#)

Figure 2: Hubble Space Telescope images of strongly lensed galaxies from our survey.



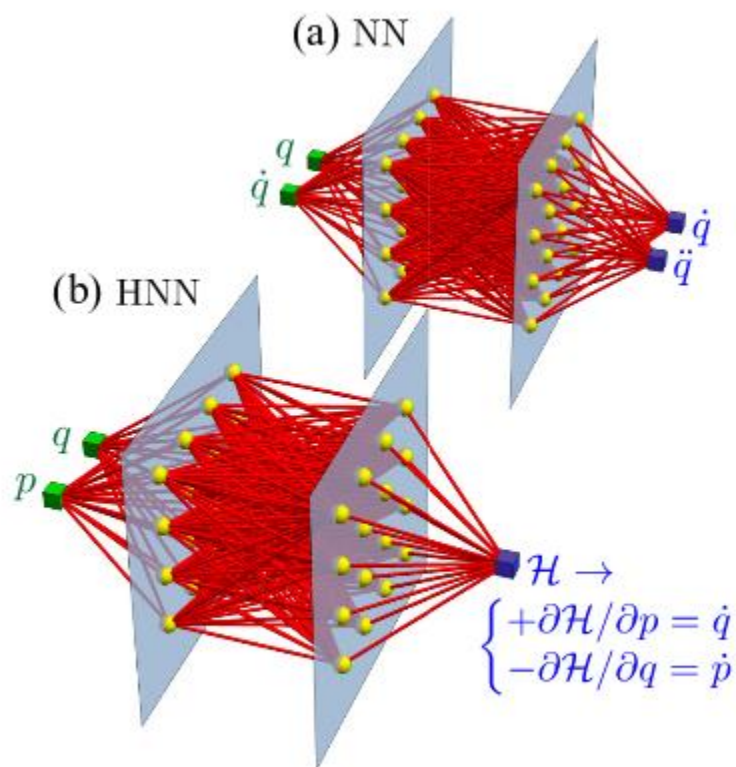
A deep Convolutional Neural Network for topology optimization with strong generalization ability

Yiquan Zhang^a · Bo Peng^a · Xiaoyi Zhou^a · Cheng Xiang^a · Dalei Wang^{a*}

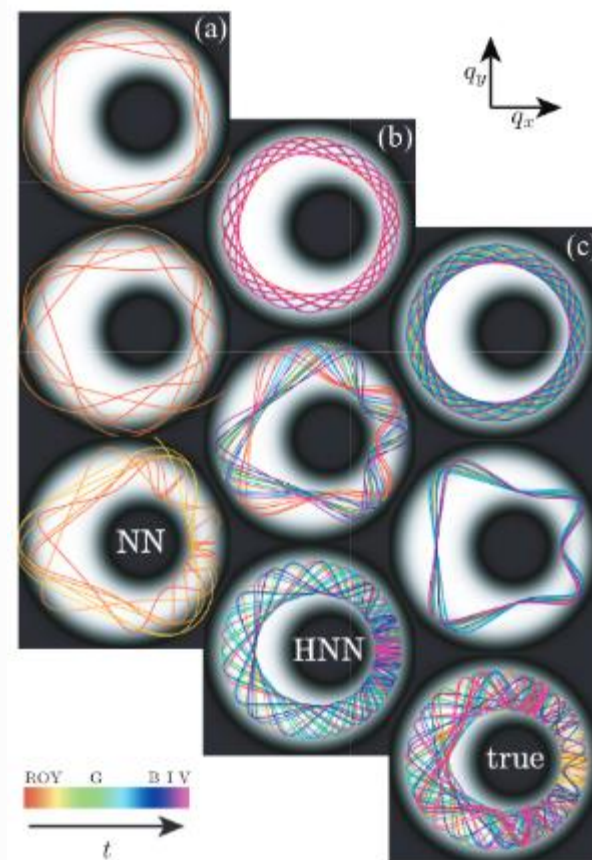


Physics-enhanced neural networks learn order and chaos

Anshul Choudhary¹, John F. Lindner^{1,2,*}, Elliott G. Holliday¹, Scott T. Miller¹, Sudeshna Sinha^{1,3}
and William L. Ditto¹



odvodi z
vzratno
propagacijo





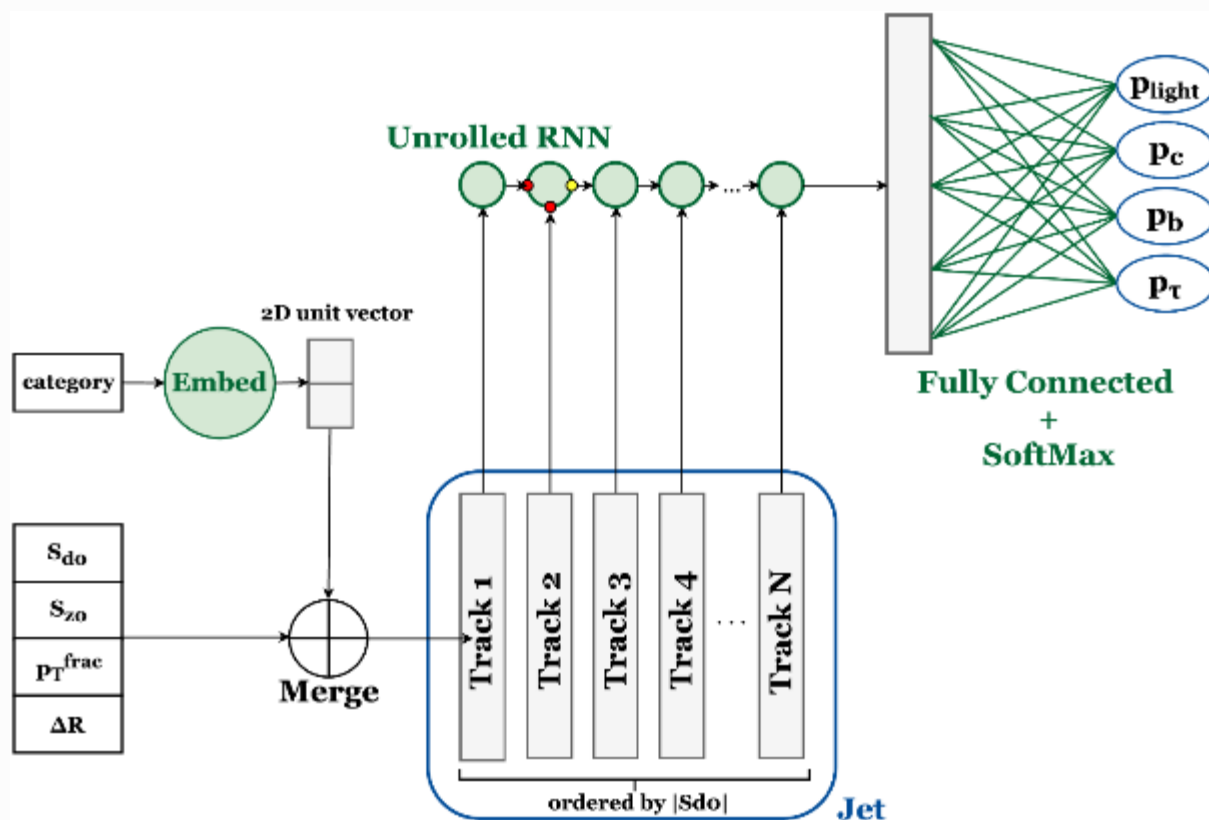
ATLAS NOTE

ATL-PHYS-PUB-2017-003

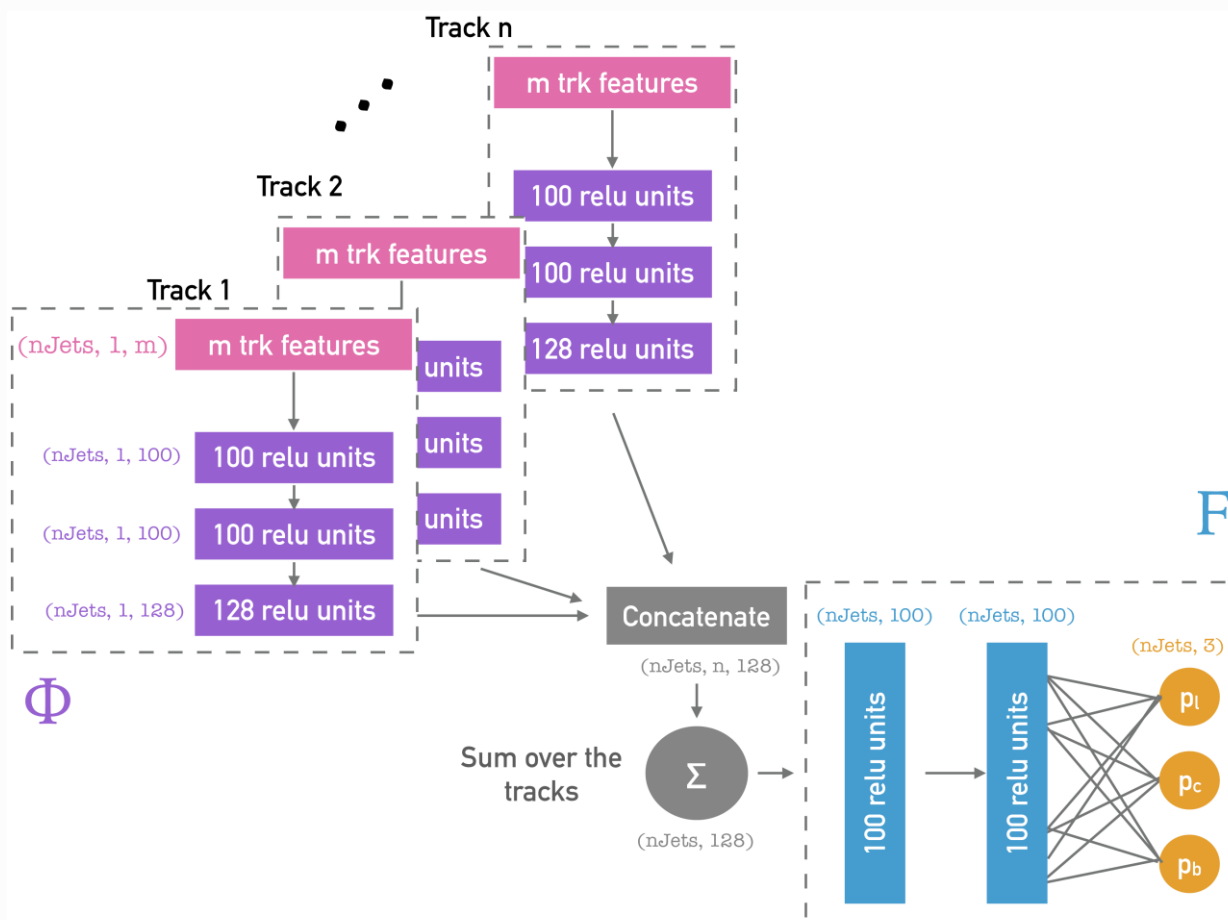
March 7, 2017



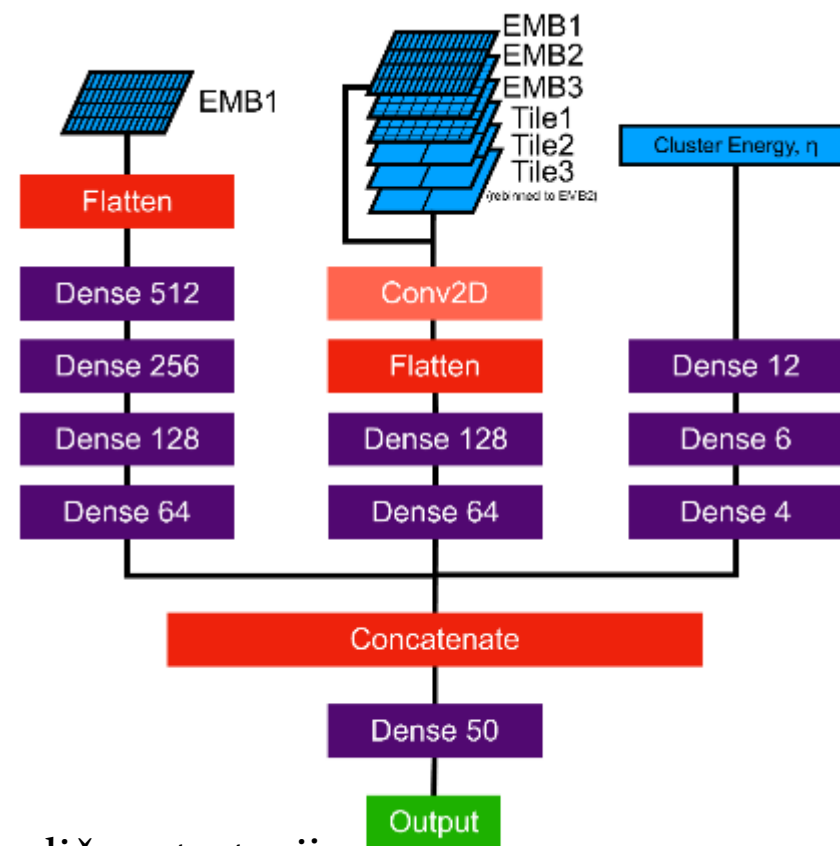
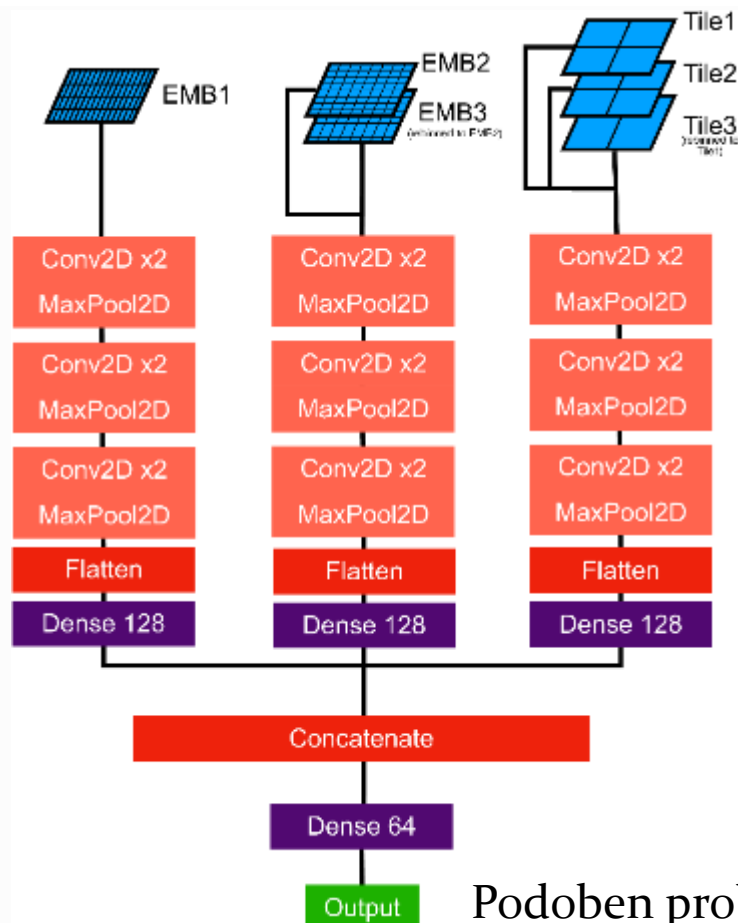
Identification of Jets Containing b -Hadrons with Recurrent Neural Networks at the ATLAS Experiment



Deep Sets based Neural Networks for Impact Parameter Flavour Tagging in ATLAS



Deep Learning for Pion Identification and Energy Calibration with the ATLAS Detector



Podoben problem, različna strategija:
konvolucija vs. tesno povezane plasti



FMF

UNIVERZA
V LJUBLJANI

Fakulteta za matematiko
in fiziko



Contents lists available at ScienceDirect

Computers & Graphics

journal homepage: www.elsevier.com/locate/cag



Multi-feature super-resolution network for cloth wrinkle synthesis

Lan Chen^{a,b}, Xiaopeng Zhang^a, Juntao Ye^{a,*}

GT



Ours



Liquid Splash Modeling with Neural Networks

Kiwon Um, Xiangyu Hu, and Nils Thuerey

Technical University of Munich, Germany

